

HFT Ops Interview Preparation

Probability / Brainteasers / Code

Gabriel Kinshuk

July 20, 2026

Contents

1	Math / Probability Theory	3
1.1	Combinatorics	3
1.1.1	Permutations	3
1.1.2	Combinations	3
1.1.3	Distributing Indistinguishable Objects (Stars and Bars)	3
1.1.4	Inclusion-Exclusion	4
1.1.5	Select Worked Problems	5
1.2	Probability	6
1.2.1	Complement	6
1.2.2	Independence	7
1.2.3	Mutual Exclusivity	7
1.2.4	Select Worked Problems	7
1.3	Expected Value	10
1.3.1	Select Worked Problems	11
1.4	Conditional Probability & Bayes	15
1.4.1	Select Worked Problems	17
1.5	Distributions	19
1.5.1	Bernoulli Distribution	20
1.5.2	Binomial Distribution	20
1.5.3	Geometric Distribution	21
1.5.4	Negative Binomial Distribution	22
1.5.5	Poisson Distribution	22
1.5.6	Exponential Distribution	23
1.5.7	Uniform Distribution	24
1.5.8	Normal (Gaussian) Distribution	24
1.5.9	Lognormal Distribution	26
1.5.10	Reference Tables	26
1.5.11	Select Worked Problems	28
2	Brainteasers	29
3	Code	31
3.1	SQL Patterns (PostgreSQL)	31
3.1.1	Filtering and Joins	31
3.1.2	Window Functions	31
3.1.3	Aggregation	34
3.1.4	NULL Handling	34

3.2	Python Patterns	36
3.3	Algorithmic Paradigms	43
3.4	Recursion and Memoization	44
3.5	Heaps and Priority Queues	47
3.6	Sliding Window	48

1 Math / Probability Theory

1.1 Combinatorics

A counting problem is pinned down by two questions. Does the order of the chosen elements matter, and may an element be chosen more than once. The formulas below are organized around those two axes, plus a correction for identical elements and two escape hatches (the complement and inclusion-exclusion) for when a direct count fans out into many cases.

1.1.1 Permutations

Definition 1.1 (Permutation). A permutation is an ordered selection. Choosing k items from n distinct options where order matters gives

$$P(n, k) = \frac{n!}{(n-k)!}.$$

The formula counts position by position. The first slot has n candidates, the second has $n-1$ once one item is placed, and so on for k slots, giving the falling product $n(n-1)\cdots(n-k+1)$. Writing that product as $\frac{n!}{(n-k)!}$ just cancels the unused tail $(n-k)!$.

Definition 1.2 (Permutations with identical elements). When the n items being arranged include groups of identical copies with frequencies $n_1, n_2, \dots, n_k$, the number of distinguishable arrangements is

$$\frac{n!}{n_1! n_2! \cdots n_k!}.$$

Start from the $n!$ arrangements that would exist if every item were distinct. Each identical group of size n_i has $n_i!$ internal orderings that all produce the same visible arrangement, so dividing by each $n_i!$ removes exactly those duplicates.

1.1.2 Combinations

Definition 1.3 (Combination). A combination is an unordered selection. Choosing k items from n distinct options where order does not matter gives

$$C(n, k) = \binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

Combinations come from permutations by discarding the ordering. Each unordered set of k items can be arranged in $k!$ ways, and $P(n, k)$ counts all of those as distinct, so

$$C(n, k) = \frac{P(n, k)}{k!}.$$

1.1.3 Distributing Indistinguishable Objects (Stars and Bars)

Theorem 1.4 (Stars and bars). *The number of ways to place n identical objects into k distinct bins, where a bin may hold any number of objects including zero, is*

$$\binom{n+k-1}{k-1} = C(n+k-1, k-1).$$

Proof. Lay the n objects in a row as stars and insert $k - 1$ dividers (bars) to cut the row into k groups. Reading left to right, objects before the first bar fall in bin 1, objects between the first and second bar fall in bin 2, and so on through bin k . Every placement of the bars produces one distribution, and every distribution corresponds to exactly one placement. The row has $n + (k - 1)$ positions, and a distribution is fixed by choosing which $k - 1$ of them hold bars, which is $\binom{n+k-1}{k-1}$. $\square$

1.1.4 Inclusion-Exclusion

Theorem 1.5 (Inclusion-Exclusion). *The size of a union is the alternating sum of the sizes of all intersections. For two and three sets,*

$$|A \cup B| = |A| + |B| - |A \cap B|,$$

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|.$$

The mechanism is bookkeeping on how many times each element gets counted. Summing the individual set sizes counts an element that lies in two sets twice, so subtracting the pairwise intersections corrects it back to one. An element in all three sets is now counted $3 - 3 = 0$ times, so adding the triple intersection restores it to one. The pattern continues to any number of sets: terms over an odd number of sets are added and terms over an even number are subtracted, which leaves every element counted exactly once no matter how many sets contain it.

Which counting tool

Order matters?	Repeats allowed?	Count
yes	no	$P(n, k) = \frac{n!}{(n-k)!}$
no	no	$C(n, k) = \binom{n}{k}$
yes	yes	n^k
no	yes	$\binom{n+k-1}{k-1}$ (stars and bars)

Two escape hatches handle counts that fan out into many subcases. Use the **complement** ($1 - P$ of the unwanted case) on “at least one” phrasing, and use **inclusion-exclusion** on “divisible by a or b ” style overlaps.

1.1.5 Select Worked Problems

Problem 1.1

How many ways can you arrange the letters in the word BANANA?

Solution. There are $n = 6$ elements. Repeated items have frequency 3 and 2 (Ns and As). Then the number of arrangements are $\frac{6!}{3! \times 2!} = \frac{720}{12} = 60$.

Problem 1.2

You roll 2 dice. How many distinct outcomes are there? What if you can't tell the dice apart?

Solution. With two dice, there are $6 \times 6 = 36$ outcomes if we can tell the dice apart. If we *can't* tell the dice apart, then we can solve either logically or formulaically:

Logically, we can see that from the original 36 combinations, there are 6 outcomes where the dice are the same, leaving 30 outcomes using mixed pairs of dice. Of these, half are duplicate when we can't tell the dice apart, leaving 15 outcomes. We can then add the 6 double outcomes back in for a total of 21 outcomes.

Formulaically, we can solve using a similar approach, taking the sum of mixed pair outcomes and double outcomes as $C(6, 2) + C(6, 1) = 15 + 6 = 21$.

Lastly, we can consider this problem using the stars and bars approach, where we notice that there are $n = 2$ identical dice and 6 distinct bins with $k = 5$ dividers between them. Then we can count the outcomes by $C(2 + 6 - 1, 6 - 1) = C(7, 5) = 21$

Problem 1.3

How many ways can you distribute 8 identical balls into 3 distinct boxes?

Solution. This is a direct application of stars and bars with $n = 8$ and $k = 3$. The number of outcomes is $C(8 + 3 - 1, 3 - 1) = C(10, 2) = \frac{10 \times 9 \times 8 \times \dots \times 3}{2} = 45$.

Problem 1.4

A deck has 52 cards. How many 5-card hands contain at least one ace?

Solution. Since this solution asks for the number of hands containing at least one ace, it will be simpler to compute the complement. This is the number of hands containing no aces, we then subtract from the total number of 5 card hands: $C(52, 5) - C(48, 5) = 886,656$.

Problem 1.5

You have 5 books on a shelf. In how many ways can you choose 3 books and arrange them left-to-right on a new shelf?

Solution. This is simply $C(5, 3) \times P(3, 3) = \frac{5 \times 4 \times 3}{3!} \times \frac{3!}{0!} = \frac{60 \times 3!}{3!} = 60$.

Problem 1.6

A pizza place offers 8 toppings. How many different pizzas can you make if you can choose any number of toppings (including none)?

Solution. We can solve this by taking the summation of $C(8, i)$ from $i = 0$ to $i = 8$:

$$\text{Total outcomes} = \sum_{i=0}^8 C(8, i) = 256$$

There is a more succinct solution we can find if we approach the problem seeing each topping as the binary choice of being either on or off of the pizza, then the total outcomes are $2^8 = 256$.

Problem 1.7

How many ways can 12 people be split into 3 groups of 4?

Solution. We choose each group independently with the remaining members of the pool, and then divide by the reorderings since the groups are indistinguishable:

$$\frac{C(12, 4) \times C(8, 4) \times C(4, 4)}{3!} = \frac{495 \times 70 \times 1}{6} = 34,650.$$

1.2 Probability

Definition 1.6 (Sample Space). The sample space Ω is the set of all possible outcomes of an experiment.

Most problems begin by fixing the sample space and then identifying the subset of outcomes that count as favourable. Getting the sample space right, which means choosing the correct unit of observation and the correct notion of “equally likely”, settles most of the difficulty before any arithmetic starts.

Definition 1.7 (Event). An event is a subset of the sample space, the collection of outcomes of interest. Its probability is the total weight of the outcomes it contains.

1.2.1 Complement

Complement

For any event A ,

$$P(A) = 1 - P(A^c),$$

where A^c is the event that A does not occur.

The complement is the first move whenever the direct event splits into many cases while its negation is a single clean case. “At least one” is the signature cue: the direct count runs over one, two, three or more successes, while the negation is the single event of zero successes. The birthday and “at least one head” problems below both collapse this way.

1.2.2 Independence

Definition 1.8 (Independence). Two events are independent when the occurrence of one leaves the probability of the other unchanged. Formally,

$$P(A \cap B) = P(A)P(B).$$

The multiplication rule is the practical test. Drawing *with* replacement keeps every draw independent because the pool is restored each time. Drawing without replacement makes the draws dependent, since removing an item changes both the size and the composition of the pool for the next draw. Independence is a statement about probabilities and is a separate idea from mutual exclusivity, which is a statement about outcomes.

1.2.3 Mutual Exclusivity

Definition 1.9 (Mutual exclusivity). Two events are mutually exclusive when they cannot both occur, so $A \cap B = \emptyset$. For such events the union probability adds:

$$P(A \cup B) = P(A) + P(B).$$

This is the special case of inclusion-exclusion where the intersection is empty. The general identity $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ carries the overlap term $P(A \cap B)$, and mutual exclusivity sets that term to zero. A common error is to add probabilities for events that can co-occur, which double counts the overlap.

1.2.4 Select Worked Problems

Problem 1.8

How many people do you need in a room for a >50% chance that two share a birthday?

Solution. Computing this directly is impractical, since there are many ways for multiple people to share a birthday. Instead, using the complement, we simply subtract the probability that *no* people share a birthday from 1:

$$P(\text{all different}) = \prod_{i=1}^n \frac{365 - i + 1}{365}$$

We multiply additional factors until the quantity drops *below* 0.5 (because then $1 - P$ is greater than 0.5). This turns out to be $n = 23$, where $P(\text{all different}) \approx 0.493$ and thus $P(\text{birthday is shared}) \approx 0.507$.

Problem 1.9

You flip a fair coin 3 times. What's the probability of getting exactly 2 heads?

Solution. Flipping a coin three times has eight outcomes. Of these, three involve exactly two heads, thus the probability is $\frac{3}{8} = 0.375$.

If we don't want to enumerate each of the eight options and count directly, we could use $C(3, 2) = 3$ to find the number of outcomes involving two heads.

Problem 1.10

Roll two dice. What's the probability the sum is 7?

Solution. There are 36 outcomes of rolling two dice. Of these, the pairs that equal 7 are $(1, 6), (2, 5), (3, 4), (4, 3), (5, 2), (6, 1)$. Then the probability is $\frac{6}{36} \approx 0.167$.

Problem 1.11

You draw 2 cards from a standard 52-card deck (without replacement). What's the probability both are hearts?

Solution. Without replacement, the draws are dependent. Then, the probability is:

$$\frac{13}{52} \times \frac{12}{51} \approx 0.0588$$

Note that *both* the numerator and the denominator shrank after we removed the first card.

We could also solve this by counting, if we divide the number of ways to pick two hearts by the total number of ways to pick two cards:

$$\frac{C(13, 2)}{C(52, 2)} = \frac{78}{1326} \approx 0.0588$$

Problem 1.12

You flip a fair coin 5 times. What's the probability of getting at least one head?

Solution. We can use the complement, the probability of getting *no* heads, subtracted from 1:

$$1 - \frac{1}{2^5} = 0.96875$$

Problem 1.13

You roll a fair die twice. What's the probability the second roll is strictly greater than the first?

Solution. There are 36 total ways to roll two dice. If the first roll is a 1, then there are 5 ways to roll higher on the second die. Similarly, there are 4 ways to roll higher if we rolled a 2, and so on.

Thus the number of favourable outcomes $1 + 2 + \cdots + 5$ or the triangle number of 5, $T_5 = 15$. The final answer is:

$$\frac{15}{36} \approx 0.417$$

Alternatively, we could solve by symmetry, calling roll 1 and 2 as A and B , respectively. If we notice that $P(A > B) = P(B < A)$, and $P(A = B) = \frac{6}{36}$, then we evenly divide the remaining outcomes by 2 to get $\frac{15}{36}$.

Problem 1.14

A box has 5 white and 3 black balls. You draw 3 without replacement. What's the probability of drawing exactly 2 white and 1 black?

Solution. The total ways to draw 3 balls are $C(8, 3) = 56$. We then find the ways to choose 2 white balls and 1 black ball individually, which is $C(5, 2)$ and $C(3, 1)$, then the answer is:

$$\frac{C(5, 2) \times C(3, 1)}{C(8, 3)} = \frac{10 \times 3}{56} \approx 0.536$$

Since we remove the balls without replacement, this problem follows the hypergeometric distribution.

Problem 1.15

You randomly pick a number from 1 to 100. What's the probability it's divisible by 3 or 5?

Solution. There are 33 numbers divisible by 3, and 20 numbers divisible by 5. The numbers divisible by both are the multiples of 15 of which there are 6 (15, 30, 45, 60, 75, 90). Thus:

$$P_{Div}(A \cup B) = \frac{33}{100} + \frac{20}{100} - \frac{6}{100} = 0.47$$

Problem 1.16

A family has 3 children. Assuming each child is equally likely to be a boy or girl, what's the probability that all 3 are the same gender?

Solution. There are eight outcomes, of those there are two in which all children are the same gender, so the answer is $\frac{2}{8} = 0.25$.

Problem 1.17

You throw 3 dice. What's the probability ALL three show different numbers?

Solution. There are $6^3 = 216$ total ways to throw three dice. After we throw the first dice, there are 5 options remaining, and 4 after the second dice. Then the answer is:

$$\frac{6 \times 5 \times 4}{6^3} = \frac{120}{216} \approx 0.556$$

Problem 1.18

You toss a fair coin until you get heads. What's the probability you need exactly 3 tosses?

Solution. We stop at the first heads, so needing exactly 3 tosses means the first two tosses are tails and the third is heads: T, T, H. Since tosses are independent and each outcome has probability $\frac{1}{2}$,

$$P(\text{exactly 3}) = \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2} = \frac{1}{8}.$$

We do not enumerate anything else, because exactly one sequence produces this event.

Remark (Two ways probability gets counted). The reason we can pin down this single case without listing every outcome is that this problem lives in a different kind of sample space than counting problems like the birthday question.

When every outcome is equally likely, probability is combinatorial. You divide favorable outcomes by total outcomes, so you have to characterize the entire sample space. Each outcome carries no probability on its own and earns weight only by being counted against the total.

When outcomes are not equally likely, each outcome carries its own probability directly, usually a product of independent step probabilities. Here the sequence TTH has probability $\frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{8}$ by itself, with no denominator to divide by. That is why one case can be solved in isolation.

These are the same axioms applied to different spaces. Counting favorable over total is the special case that appears when all per-outcome probabilities are equal, so the sum of probabilities collapses into a plain count. The coin problem is the general version, where the per-outcome probabilities differ and you take the one you want.

Problem 1.19

In a room of 5 people, what's the probability that at least two share a birthday? (Assume 365 days, equally likely.)

Solution. Again, we use the complement to find 1 minus the probability that no one shares a birthday:

$$1 - \left(\frac{365}{365} \times \frac{364}{365} \times \frac{363}{365} \times \frac{362}{365} \times \frac{361}{365} \right) \approx 1 - 0.9729 \approx 0.0271$$

1.3 Expected Value

Definition 1.10 (Expected Value). The expected value of a random variable is the probability-weighted average of its outcomes:

$$\mathbb{E}[X] = \sum_i x_i P(X = x_i).$$

It is the long-run mean over many repetitions. The expected value need not be an attainable outcome (a fair die has $\mathbb{E}[X] = 3.5$) and need not be the most likely one.

Linearity of expectation

For any random variables X and Y and any scalars a, b ,

$$\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y].$$

This holds with no independence assumption. Paired with indicator variables it is the highest-yield tool for expectation brainteasers, since it turns a hard expected count into a sum of easy probabilities.

Definition 1.11 (Indicator Random Variable). An indicator $\mathbf{1}\{A\}$ equals 1 when event A occurs and 0 otherwise. Its expectation is the event's probability,

$$\mathbb{E}[\mathbf{1}\{A\}] = 1 \cdot P(A) + 0 \cdot P(A^c) = P(A).$$

The variable itself is trivial. The leverage is the decomposition it enables. To find the expected *count* of how many events in a family occur, write that count as a sum of indicators, one per event, then apply linearity:

$$\mathbb{E}\left[\sum_i \mathbf{1}\{A_i\}\right] = \sum_i \mathbb{E}[\mathbf{1}\{A_i\}] = \sum_i P(A_i).$$

Because linearity ignores dependence, this works even when the events are entangled, which is what collapses an otherwise messy count into a sum of easy probabilities. The shuffle and fixed-point problems below are the standard instances.

Theorem 1.12 (Law of Total Expectation). *If the cases $B_1, \dots, B_n$ partition the sample space, meaning they are mutually exclusive and cover every outcome, then*

$$\mathbb{E}[X] = \sum_{i=1}^n \mathbb{E}[X \mid B_i] \cdot P(B_i).$$

This regroups the definition of expected value by cases. Rather than averaging over individual outcomes, average the conditional mean inside each case and weight it by the case probability. Expanding each $\mathbb{E}[X \mid B_i]$ recovers the flat sum, so the identity adds no new information, but it changes what is computable. It is the tool of choice in three situations:

- you are handed conditional averages and case probabilities rather than the full outcome distribution
- X is continuous, so the flat sum $\sum_x x P(X = x)$ does not apply while conditioning on discrete cases still does
- $\mathbb{E}[X \mid B_i]$ refers back to $\mathbb{E}[X]$ itself, which turns the conditioning into a solvable equation

The cases must be disjoint and exhaustive. Overlapping or missing cases break the weighting.

Example 1.13 (Conditioning on cases). A commute time depends on the weather. Sunny days are 60% of days with a 20 minute average, and rainy days are 40% with a 35 minute average. The two cases partition every day, so

$$\mathbb{E}[\text{commute}] = 0.6(20) + 0.4(35) = 12 + 14 = 26 \text{ minutes.}$$

Example 1.14 (Recursive expectation). Take the expected number of rolls of a die until the first 6, and call it E . One roll always happens. With probability $\frac{1}{6}$ it is a 6 and the process stops, and with probability $\frac{5}{6}$ it is not, which leaves the same problem ahead with the same expectation E :

$$E = 1 + \frac{5}{6}E \longrightarrow \frac{1}{6}E = 1 \longrightarrow E = 6.$$

This is the mean of the geometric distribution, the expected number of trials until the first success, which is $\frac{1}{p}$ with $p = \frac{1}{6}$.

1.3.1 Select Worked Problems

Problem 1.20

A game costs \$1. You flip a coin: heads you win \$3, tails you win nothing. What's $\mathbb{E}[\text{Profit}]$?

Solution. $EV = 0.5 \cdot 2 + 0.5 \cdot -1 = +0.5$

Problem 1.21

Roll a die. If it shows 6, you roll again and add. Otherwise, your score is the number shown. What's the expected score?

Solution.

$$EV = 3.5 + \frac{1}{6} \cdot EV \longrightarrow \frac{5}{6} \cdot EV = 3.5 \longrightarrow EV = 4.2$$

Problem 1.22

Shuffle a standard deck. What's the expected number of cards that are in their "correct" position (card k is in position k)?

Solution. This is linearity of expectation applied to indicators. Let $I_k = 1$ if card k lands in position k , and 0 otherwise. In a random shuffle each card is equally likely to occupy any of the 52 positions, so $\mathbb{E}[I_k] = P(I_k = 1) = \frac{1}{52}$. The number of correctly placed cards is $M = \sum_{k=1}^{52} I_k$, and by linearity

$$\mathbb{E}[M] = \sum_{k=1}^{52} \mathbb{E}[I_k] = 52 \cdot \frac{1}{52} = 1.$$

The I_k are dependent, but linearity does not require independence, so that never matters.

Problem 1.23

You flip a coin until you get heads. How many flips do you expect?

Solution.

$$EV = 1 + 0.5 \cdot EV \longrightarrow 0.5 \cdot EV = 1 \longrightarrow EV = 2$$

Problem 1.24

A cereal box contains 1 of 4 different toys (equally likely). How many boxes must you buy to expect to collect all 4?

Solution. The total number of boxes is a sum of four phase waits, so we use linearity of expectation. In phase k we already hold $k - 1$ toys and buy boxes until a new one appears. The per-trial success probability is $p = \frac{\text{favourable}}{\text{total}}$, the number of missing toys over 4. We want the reciprocal, the expected number of trials per success, which is $\frac{1}{p}$:

$$\begin{aligned} 0 \text{ toys, } p &= \frac{4}{4} \longrightarrow \mathbb{E}[T_1] = \frac{1}{p} = 1 \\ 1 \text{ toy, } p &= \frac{3}{4} \longrightarrow \mathbb{E}[T_2] = \frac{1}{p} = \frac{4}{3} \\ 2 \text{ toys, } p &= \frac{2}{4} \longrightarrow \mathbb{E}[T_3] = \frac{1}{p} = 2 \\ 3 \text{ toys, } p &= \frac{1}{4} \longrightarrow \mathbb{E}[T_4] = \frac{1}{p} = 4 \end{aligned}$$

$$\mathbb{E}[T] = \mathbb{E}[T_1] + \mathbb{E}[T_2] + \mathbb{E}[T_3] + \mathbb{E}[T_4] = 1 + \frac{4}{3} + 2 + 4 = \frac{25}{3} \approx 8.33$$

Problem 1.25

I roll a die repeatedly and keep a running sum. What's the expected number of rolls until the sum first reaches or exceeds 10?

Solution. We solve recursively. Let $\mathbb{E}[s]$ be the expected number of additional rolls needed to first reach a sum of at least 10, starting from current sum s . The answer we want is $\mathbb{E}[0]$.

Base case: $\mathbb{E}[s] = 0$ for all $s \geq 10$, since the target is already met. For $s < 10$, one roll costs 1 and moves to $s + 1, \dots, s + 6$ with equal probability $\frac{1}{6}$:

$$\mathbb{E}[s] = 1 + \frac{1}{6} \sum_{j=1}^6 \mathbb{E}[s + j], \quad \mathbb{E}[s] = 0 \text{ for } s \geq 10.$$

Working downward from the base case:

$$\begin{aligned} \mathbb{E}[9] &= 1 + \frac{1}{6}(\mathbb{E}[10] + \dots + \mathbb{E}[15]) = 1 \\ \mathbb{E}[8] &= 1 + \frac{1}{6}\mathbb{E}[9] = \frac{7}{6} \\ \mathbb{E}[7] &= 1 + \frac{1}{6}(\mathbb{E}[8] + \mathbb{E}[9]) = \frac{49}{36} \\ &\vdots \\ \mathbb{E}[0] &\approx 3.6 \end{aligned}$$

Problem 1.26

You roll a fair six-sided die. After seeing the result, you may keep it as your score, or reroll. If you reroll, the new value replaces the old one (you cannot go back to a discarded roll). You are allowed up to three rolls total, and your score is the value you are holding when you stop. Playing optimally to maximize your expected score, what is that expected score, and what is the stopping rule at each stage?

Solution. Let $\mathbb{E}[r]$ be the expected score with r rerolls remaining. Solve backwards.

With no rerolls left, you keep whatever you roll, so $\mathbb{E}[0] = 3.5$.

With one reroll left, you keep the current face only if it beats the continuation value $\mathbb{E}[0] = 3.5$, i.e. keep on $\{4, 5, 6\}$ and reroll otherwise:

$$\mathbb{E}[1] = \frac{1}{6}(4) + \frac{1}{6}(5) + \frac{1}{6}(6) + \frac{3}{6}(3.5) = \frac{17}{4} = 4.25.$$

With two rerolls left, the continuation value is now $\mathbb{E}[1] = 4.25$, so you keep only faces that beat it, $\{5, 6\}$, and reroll on $\{1, 2, 3, 4\}$:

$$\mathbb{E}[2] = \frac{1}{6}(6) + \frac{1}{6}(5) + \frac{4}{6}(4.25) = \frac{14}{3} \approx 4.67.$$

Problem 1.27

You pick a random number uniformly from $[0, 1]$. Then you pick another. Then another. You stop when the running total exceeds 1. How many numbers do you expect to pick?

Solution. Let N be the number of draws until the running sum first exceeds 1, and let $X_1, X_2, \dots$ be the draws. We want $\mathbb{E}[N]$.

Step 1: a tail probability. The *tail probability* $\mathbb{P}(N > n)$ is the chance N exceeds a cutoff n , i.e. the chance we still need more draws after n of them. Needing more than n draws means the first n have not yet crossed 1:

$$N > n \iff X_1 + \dots + X_n \leq 1.$$

Step 2: geometry The point $(X_1, \dots, X_n)$ lands uniformly in the unit hypercube $[0, 1]^n$, which has volume 1. Since the draws are uniform, the probability of landing in any region equals that region's volume. The region where the coordinates are nonnegative and sum to at most 1 is a *simplex* (the n -dimensional triangle: a segment in 1D, triangle in 2D, tetrahedron in 3D). Its volume is $\frac{1}{n!}$ ($V_n(t) = \int_0^t V_{n-1}(t-x) dx$, starting from $V_1(t) = t$. Then $V_n(t) = \frac{t^n}{n!}$, evaluated at $t = 1$, $V_n(1) = \frac{1}{n!}$), so

$$\mathbb{P}(N > n) = \mathbb{P}(X_1 + \dots + X_n \leq 1) = \frac{1}{n!}.$$

Step 3: sum the tails. For a nonnegative integer variable, the mean equals the sum of its tail probabilities:

$$\mathbb{E}[N] = \sum_{n=0}^{\infty} \mathbb{P}(N > n).$$

This holds because an outcome $N = k$ clears the cutoffs $n = 0, 1, \dots, k-1$, so it is counted exactly k times across the sum, contributing its own value. Substituting the tails:

$$\mathbb{E}[N] = \sum_{n=0}^{\infty} \frac{1}{n!} = \frac{1}{0!} + \frac{1}{1!} + \frac{1}{2!} + \dots = e \approx 2.718.$$

1.4 Conditional Probability & Bayes

Definition 1.15 (Conditional Probability). The probability of A given that B has occurred is

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0.$$

Conditioning on B restricts attention to the outcomes inside B and rescales their probabilities to sum to 1, which is the role of the denominator.

Theorem 1.16 (Bayes' Theorem). *Bayes' theorem reverses the direction of conditioning, updating a hypothesis as evidence arrives:*

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}.$$

The four pieces are worth memorizing by name:

- $P(A | B)$ is the posterior, the updated belief in A after seeing B
- $P(B | A)$ is the likelihood, how well A explains the evidence
- $P(A)$ is the prior, the belief in A before the evidence
- $P(B)$ is the marginal, the total probability of the evidence

Odds form skips the marginal

Dividing Bayes' theorem for two competing hypotheses cancels the shared marginal $P(B)$:

$$\underbrace{\frac{P(A | B)}{P(A^c | B)}}_{\text{posterior odds}} = \underbrace{\frac{P(B | A)}{P(B | A^c)}}_{\text{likelihood ratio}} \times \underbrace{\frac{P(A)}{P(A^c)}}_{\text{prior odds}}.$$

Each piece of evidence multiplies the prior odds by its likelihood ratio, so repeated independent evidence just multiplies again. This avoids computing $P(B)$ and is the fastest route through two-hypothesis update problems.

Theorem 1.17 (Law of Total Probability). *If $B_1, B_2, \dots, B_n$ partition the sample space (mutually exclusive and exhaustive), then*

$$P(A) = \sum_i P(A | B_i) P(B_i).$$

This is how the marginal in Bayes' theorem is usually assembled. When $P(A)$ is not known directly but the conditional probabilities on each case are, summing those conditionals weighted by the case probabilities builds it. The two-way form $P(A) = P(A | C)P(C) + P(A | C^c)P(C^c)$ covers most interview uses.

Example 1.18 (Monty Hall Problem). Assume the contestant chooses Door 1 and Monty opens Door 2 to reveal a goat. Let A_i be the event the car is behind Door i (prior $P(A_i) = \frac{1}{3}$), and B be the event Monty opens Door 2.

The likelihoods $P(B|A_i)$ are:

- $P(B|A_1) = \frac{1}{2}$ (Monty randomly picks Door 2 or 3)
- $P(B|A_2) = 0$ (Monty cannot reveal the car)
- $P(B|A_3) = 1$ (Monty is forced to open Door 2)

The marginal probability of B is:

$$P(B) = \sum_{i=1}^3 P(B|A_i)P(A_i) = \left(\frac{1}{2}\right)\left(\frac{1}{3}\right) + (0)\left(\frac{1}{3}\right) + (1)\left(\frac{1}{3}\right) = \frac{1}{2}$$

By Bayes' Theorem, the posterior probabilities of winning by staying (A_1) or switching (A_3) are:

$$P(A_1|B) = \frac{P(B|A_1)P(A_1)}{P(B)} = \frac{\left(\frac{1}{2}\right)\left(\frac{1}{3}\right)}{\frac{1}{2}} = \frac{1/6}{1/2} = \frac{1}{3}$$
$$P(A_3|B) = \frac{P(B|A_3)P(A_3)}{P(B)} = \frac{(1)\left(\frac{1}{3}\right)}{\frac{1}{2}} = \frac{1/3}{1/2} = \frac{2}{3}$$

Switching doubles the probability of winning.

1.4.1 Select Worked Problems

Problem 1.28

I roll two dice. Given that the sum is 8, what's the probability that one of the dice shows a 3?

Solution. The possible ways to roll a sum of 8 are: (2, 6), (3, 5), (4, 4), (5, 3), (6, 2). Of these, two options have a 3 in them, so the answer is $\frac{2}{5} = 0.4$.

Problem 1.29

A bag has 3 red and 2 blue balls. You draw one, don't replace it, then draw another. What's $P(\text{2nd is red} \mid \text{1st was blue})$?

Solution. Once we drew one blue on, the remaining bag composition is 3 red balls and 1 blue one. Then, there is a $\frac{3}{4}$ probability that the second will be red.

Problem 1.30

60% of emails are spam. A filter catches 95% of spam but also flags 3% of legitimate emails. An email is flagged. What's the probability it's actually spam?

Solution. We want $P(\text{Spam} \mid \text{Flagged})$. The likelihood $P(\text{Flagged} \mid \text{Spam})$ is given as 0.95, the prior $P(\text{Spam})$ is 0.6, and the marginal probability $P(\text{Flagged})$ is $0.95 \cdot 0.6 + 0.03 \cdot 0.4 = 0.582$.

Then the posterior is

$$P(\text{Spam} \mid \text{Flagged}) = \frac{P(\text{Flagged} \mid \text{Spam}) \times P(\text{Spam})}{P(\text{Flagged})} = \frac{0.95 \times 0.6}{0.582} \approx 0.979$$

Problem 1.31

You have 2 coins. Coin A is fair (50% heads). Coin B is biased (75% heads). You pick a coin at random and flip it. It lands heads. What's the probability you picked coin B?

Solution. We are looking for the posterior, $P(B \mid H)$.

The priors are $P(A) = P(B) = 0.5$. The likelihoods are $P(H \mid A) = 0.5$ and $P(H \mid B) = 0.75$. Then $P(H) = P(H \mid A) \cdot P(A) + P(H \mid B) \cdot P(B) = 0.5 \cdot 0.5 + 0.75 \cdot 0.5 = 0.625$. Then finally

$$P(B \mid H) = \frac{P(H \mid B) \cdot P(B)}{P(H)} = \frac{0.75 \cdot 0.5}{0.625} = 0.6$$

After flipping and landing on heads, our belief that we picked coin B increased from 0.5 to 0.6.

Problem 1.32

Same as 1.31. The chosen coin is flipped twice and get heads both times. Now what's $P(B)$?

Solution. The only difference is our priors are $P(A) = 0.4$ and $P(B) = 0.6$. Then we update $P(H) = P(H \mid A) \cdot P(A) + P(H \mid B) \cdot P(B) = 0.5 \cdot 0.4 + 0.75 \cdot 0.6 = 0.65$. Finally:

$$P(B \mid H) = \frac{P(H \mid B) \cdot P(B)}{P(H)} = \frac{0.75 \cdot 0.6}{0.65} = \frac{9}{13} \approx 0.692$$

Problem 1.33

A family has 2 children. Given that at least one child is a boy, what's the probability both are boys?

Solution. We can answer this by considering the restriction over the sample space caused by the new information. Originally, for two children there would be four possibilities, of which 1 has two boys. After we know that at least one child is a boy, only three options remain $(B, G), (B, B), (G, B)$. Thus, the probability both are boys is $\frac{1}{3}$.

Problem 1.34

Three cards: one is red on both sides, one is blue on both sides, one is red on one side and blue on the other. You pick a card at random and see a red side. What's the probability the other side is also red?

Solution. The natural first instinct is to treat the cards as the sample space, as with the births problem: restrict to cards consistent with seeing a red face, then count outcomes among those. This fails here because a card is not a single unit of observation. When a card has two sides of the same color, "seeing red" can happen in more than one way for that card. There's no such internal multiplicity in the births problem.

So take the six sides, not the three cards, as the equally likely outcomes. Three of these six are red: two belong to the RR card, one belongs to the RB card. Since each red side was equally likely to be the one we observed, and two of the three red sides have a red back:

$$P(\text{other side red} \mid \text{observed red}) = \frac{2}{3}.$$

Problem 1.35

You have a 6-sided die. You don't know if it's fair or loaded. A fair die shows each number with probability $1/6$. The loaded die shows 6 with probability $1/2$ and each other number with probability $1/10$. You pick a die at random (50-50) and roll a 6. What's $P(\text{loaded})$?

Solution. By Bayes' theorem, with $P(\text{loaded}) = P(\text{fair}) = 0.5$:

$$P(\text{loaded} \mid 6) = \frac{P(6 \mid \text{loaded}) P(\text{loaded})}{P(6)} = \frac{0.5 \times 0.5}{0.5 \times 0.5 + \frac{1}{6} \times 0.5} = \frac{0.25}{1/3} = 0.75$$

Alternatively, work in odds instead of probabilities. Dividing the Bayes' theorem expressions for $P(\text{loaded} \mid 6)$ and $P(\text{fair} \mid 6)$ cancels $P(6)$ from both, leaving:

$$\underbrace{\frac{P(\text{loaded} \mid 6)}{P(\text{fair} \mid 6)}}_{\text{posterior odds}} = \underbrace{\frac{P(6 \mid \text{loaded})}{P(6 \mid \text{fair})}}_{\text{likelihood ratio}} \times \underbrace{\frac{P(\text{loaded})}{P(\text{fair})}}_{\text{prior odds}}$$

So the posterior odds are just the prior odds scaled by the likelihood ratio, no need to compute $P(6)$ at all. Here, the prior odds are $\frac{P(\text{loaded})}{P(\text{fair})} = \frac{0.5}{0.5} = 1$, and the likelihood ratio is $\frac{P(6 \mid \text{loaded})}{P(6 \mid \text{fair})} = \frac{0.5}{1/6} = 3$. So the posterior odds are $1 \times 3 = 3$, i.e. $3 : 1$ in favor of loaded, giving $P(\text{loaded} \mid 6) = 3/4 = 0.75$.

In short, a six is three times more likely to appear if the die is loaded, so each 6 observed multiplies the odds by 3. If we observed *another* 6, the odds would update again: $3 : 1 \rightarrow 9 : 1$, giving $P(\text{loaded}) = \frac{9}{10}$.

1.5 Distributions

A named distribution is a compressed description of a random process. What makes the catalogue useful is not the formulas in isolation but the mapping from a verbal cue in the problem statement to a process, and from that process to a mean, a variance, and a tail. The organizing question for any problem is therefore which process generated the randomness rather than which formula to apply.

Definition 1.19 (Random variable and distribution). A random variable X is a function assigning a number to each outcome in the sample space Ω . Its distribution specifies how probability is spread across the values X can take, and it is described by one of three equivalent objects:

- the probability mass function (PMF) $P(X = x)$ in the discrete case, with $\sum_x P(X = x) = 1$
- the probability density function (PDF) $f(x)$ in the continuous case, with $\int_{-\infty}^{\infty} f(x) dx = 1$
- the cumulative distribution function (CDF) $F(x) = P(X \leq x)$, defined in both cases

The set of values with nonzero probability is the support.

In the continuous case $P(X = x) = 0$ for every single point, so density is not probability. Only integrals of the density over an interval carry probability, which is why continuous questions are answered through the CDF as $P(a < X \leq b) = F(b) - F(a)$.

Recognize the process first

A named distribution (Bernoulli, Binomial, Geometric, Poisson, and so on) is a family arising from a common underlying random process, indexed by one or more parameters such as n and p for the Binomial. Once a problem is matched to its process, the PMF or PDF, mean, and variance follow immediately from the known formulas for the family. Three cues carry most of the discrete cases:

- a *fixed* number of trials with a counted number of successes points to the Binomial
- a *fixed* number of successes with a counted number of trials points to the Geometric or Negative Binomial
- a count over a window of time or space with no natural trial count points to the Poisson

Definition 1.20 (Moment). The n -th moment of a random variable X is $\mathbb{E}[X^n]$, and its n -th central moment is $\mathbb{E}[(X - \mu)^n]$, taken about the mean $\mu = \mathbb{E}[X]$. A moment exists only when the defining expectation converges absolutely, meaning $\mathbb{E}[|X|^n] < \infty$.

Moments are summary numbers describing the shape of a distribution, each one weighting outcomes by a higher power and so responding more strongly to values far from the center. The first four carry standard names and interpretations:

- the first moment $\mathbb{E}[X]$ is the mean, locating the distribution
- the second central moment $\mathbb{E}[(X - \mu)^2]$ is the variance, measuring spread
- the third standardized moment is the skewness, measuring asymmetry
- the fourth standardized moment is the kurtosis, measuring tail weight

Standardizing divides the central moment by σ^n , which makes skewness and kurtosis dimensionless and therefore comparable across distributions. A distribution is not guaranteed to possess any given moment, since sufficiently heavy tails make the integral diverge, and moments fail from the top down in the sense that a finite n -th moment forces every lower moment to be finite as well.

1.5.1 Bernoulli Distribution

Definition 1.21 (Bernoulli). A random variable X follows a Bernoulli distribution with parameter $p \in [0, 1]$, written $X \sim \text{Bernoulli}(p)$, if it takes exactly two values:

$$P(X = 1) = p, \quad P(X = 0) = 1 - p.$$

It models a single trial with two outcomes, conventionally labeled success (1) and failure (0).

$$\mu = p \quad \sigma^2 = p(1 - p)$$

Both moments follow in one line. Since X only takes the values 0 and 1, the identity $X^2 = X$ holds pointwise, so $\mathbb{E}[X^2] = \mathbb{E}[X] = p$ and

$$\text{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2 = p - p^2 = p(1 - p).$$

The variance is largest at $p = \frac{1}{2}$, where the outcome is most uncertain, and shrinks to zero as p approaches 0 or 1, where the result becomes nearly certain.

Building block for other distributions

The Bernoulli is rarely the distribution of interest on its own. It is the atomic unit that other named distributions are built from. A Binomial random variable is a sum of n i.i.d. $\text{Bernoulli}(p)$ trials, and a Geometric random variable counts trials until the first Bernoulli success. Recognizing a Bernoulli inside a larger problem, often through an indicator variable $\mathbf{1}\{\cdot\}$, is frequently the first step in applying linearity of expectation to more complex sums.

1.5.2 Binomial Distribution

Definition 1.22 (Binomial). $X \sim \text{Binomial}(n, p)$ counts the number of successes in n independent trials, each succeeding with the same probability p . Its PMF is

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}, \quad k = 0, 1, \dots, n.$$

Proof. Fix one particular sequence of outcomes with k successes and $n - k$ failures. Independence makes its probability the product $p^k (1 - p)^{n-k}$, and that value depends only on the counts, not on the positions. The number of distinct sequences with k successes is the number of ways to choose which trials succeed, namely $\binom{n}{k}$. Summing the common probability over those disjoint sequences gives the stated PMF. $\square$

$$\mu = np \quad \sigma^2 = np(1 - p)$$

Writing $X = \sum_{i=1}^n X_i$ as a sum of i.i.d. $\text{Bernoulli}(p)$ variables gives both moments without touching the PMF. Linearity of expectation yields $\mathbb{E}[X] = np$ with no independence needed. Variance requires independence, which zeroes every covariance cross term and leaves $\text{Var}(X) = n \cdot p(1 - p)$.

Sampling with replacement against without

The Binomial requires that p stay constant across trials, which holds when sampling with replacement. Sampling without replacement changes the composition of the pool after each draw and produces the Hypergeometric distribution instead. When the population is large relative to the sample, the shift in p is negligible and the Binomial is the standard approximation.

Definition 1.23 (Hypergeometric). Drawing n items without replacement from a population of N containing K successes, the number of successes drawn has PMF

$$P(X = k) = \frac{\binom{K}{k} \binom{N-K}{n-k}}{\binom{N}{n}}.$$

Its mean is $\mu = n \frac{K}{N}$, matching the Binomial with $p = K/N$, and its variance is

$$\sigma^2 = np(1-p) \frac{N-n}{N-1},$$

where the factor $\frac{N-n}{N-1}$ is the finite population correction.

The mean is unchanged because linearity of expectation ignores the dependence between draws. The variance is lower because the draws are negatively correlated: each draw removes an item from the pool, so a success on one draw leaves fewer successes available and makes a success on the next slightly less likely. That downward pull is exactly the correction factor, which is below 1, so sampling without replacement is strictly less variable than sampling with replacement. It collapses to 1 as N grows relative to n , which recovers the Binomial.

1.5.3 Geometric Distribution

Definition 1.24 (Geometric). $X \sim \text{Geometric}(p)$ counts the number of independent trials up to and including the first success, where each trial succeeds with probability p . Its PMF is

$$P(X = k) = (1-p)^{k-1}p, \quad k = 1, 2, 3, \dots$$

The PMF is a direct product. Reaching the first success on trial k requires $k-1$ failures followed by one success, and independence multiplies those probabilities. The tail probability is equally compact, since needing more than k trials means the first k all failed:

$$P(X > k) = (1-p)^k.$$

$$\mu = \frac{1}{p} \quad \sigma^2 = \frac{1-p}{p^2}$$

Two conventions

Some texts define the Geometric as the number of *failures before* the first success, with support $k = 0, 1, 2, \dots$ and PMF $(1-p)^k p$. That version has mean $\frac{1-p}{p}$, exactly one less, while the variance $\frac{1-p}{p^2}$ is identical since shifting a variable by a constant does not change its spread. The two differ by the single trial on which the success occurs, so the convention in force must be established before any numerical comparison. The trials convention with mean $\frac{1}{p}$ is the one used throughout these notes.

Theorem 1.25 (Memorylessness). *The Geometric is the only discrete distribution with the memoryless property:*

$$P(X > m + n \mid X > m) = P(X > n).$$

Past failures carry no information about how many further trials are required.

Proof. Apply the definition of conditional probability. The event $\{X > m + n\}$ is contained in $\{X > m\}$, so their intersection is the former:

$$P(X > m + n \mid X > m) = \frac{P(X > m + n)}{P(X > m)} = \frac{(1 - p)^{m+n}}{(1 - p)^m} = (1 - p)^n = P(X > n). \quad \square$$

Memorylessness is the reason the recursive expectation argument works. A coin that has landed tails ten times in a row still needs $\frac{1}{p}$ further flips in expectation, so conditioning on the first trial reproduces the original problem and gives $\mathbb{E}[X] = 1 + (1 - p)\mathbb{E}[X]$, which solves to $\frac{1}{p}$.

1.5.4 Negative Binomial Distribution

Definition 1.26 (Negative Binomial). $X \sim \text{NegBin}(r, p)$ counts the number of independent trials up to and including the r -th success. Its PMF is

$$P(X = k) = \binom{k-1}{r-1} p^r (1-p)^{k-r}, \quad k = r, r+1, \dots$$

The binomial coefficient counts the arrangements of the first $r-1$ successes among the first $k-1$ trials. Trial k is forced to be a success, which is what pins the r -th success to the final position rather than anywhere in the sequence.

$$\mu = \frac{r}{p} \quad \sigma^2 = \frac{r(1-p)}{p^2}$$

Both follow from writing $X = G_1 + G_2 + \dots + G_r$, a sum of r independent $\text{Geometric}(p)$ waiting times. Memorylessness is what guarantees each waiting period restarts fresh after a success, so the G_i are i.i.d. The mean is r times the Geometric mean by linearity of expectation. The variance is r times the Geometric variance by *independence*, not by linearity, since variances add only when covariance terms vanish.

1.5.5 Poisson Distribution

Definition 1.27 (Poisson). $X \sim \text{Poisson}(\lambda)$ counts discrete events occurring in a fixed window of time or space, when events arrive independently at a constant average rate λ per window. Its PMF is

$$P(X = k) = e^{-\lambda} \frac{\lambda^k}{k!}, \quad k = 0, 1, 2, \dots$$

$$\mu = \lambda \quad \sigma^2 = \lambda$$

The equality of mean and variance serves as a diagnostic. Count data whose sample variance materially exceeds its mean is overdispersed and is not Poisson. This occurs routinely in market data, where order arrivals cluster rather than arriving at a constant rate, and it motivates the mixture and doubly stochastic models that generalize the Poisson.

Theorem 1.28 (Poisson limit of the Binomial). *Fix $\lambda > 0$ and let $n \rightarrow \infty$ with $p = \lambda/n$, so that the expected count $np = \lambda$ stays constant. Then*

$$\binom{n}{k} p^k (1-p)^{n-k} \rightarrow e^{-\lambda} \frac{\lambda^k}{k!}.$$

Proof. Substitute $p = \lambda/n$ and split the expression into three factors:

$$\underbrace{\frac{n(n-1)\cdots(n-k+1)}{n^k}}_{\rightarrow 1} \cdot \frac{\lambda^k}{k!} \cdot \underbrace{\left(1 - \frac{\lambda}{n}\right)^n}_{\rightarrow e^{-\lambda}} \cdot \underbrace{\left(1 - \frac{\lambda}{n}\right)^{-k}}_{\rightarrow 1}.$$

The first factor is a ratio of k terms each approaching n , the third is the standard limit defining $e^{-\lambda}$, and the fourth has a fixed exponent with a base approaching 1. The surviving product is the Poisson PMF. $\square$

The interpretation matters more than the algebra. The Poisson is the law of rare events, meaning many opportunities for an event to occur with a small probability at each one. This is why it models message counts, quote updates, and defaults, where no natural trial count exists but a rate does.

Superposition and thinning

Superposition. Independent Poisson streams merge into a Poisson stream with the summed rate, so $X \sim \text{Poisson}(\lambda_1)$ and $Y \sim \text{Poisson}(\lambda_2)$ independent give $X + Y \sim \text{Poisson}(\lambda_1 + \lambda_2)$. Combining order flow across venues is the standard application.

Thinning. If each event in a $\text{Poisson}(\lambda)$ stream is independently retained with probability q , the retained events form a $\text{Poisson}(\lambda q)$ stream, and the discarded ones form an independent $\text{Poisson}(\lambda(1-q))$ stream. Filtering a stream to buy orders only is the standard application.

1.5.6 Exponential Distribution

Definition 1.29 (Exponential). $X \sim \text{Exponential}(\lambda)$ is the continuous waiting time until the next event in a Poisson process of rate λ . Its density and CDF are

$$f(x) = \lambda e^{-\lambda x}, \quad F(x) = 1 - e^{-\lambda x}, \quad x \geq 0.$$

$$\mu = \frac{1}{\lambda} \quad \sigma^2 = \frac{1}{\lambda^2}$$

The Exponential is the continuous counterpart of the Geometric, and the pairing runs deep. The Geometric counts discrete trials until a success while the Exponential measures continuous time until an arrival, and each is the unique memoryless distribution on its domain:

$$P(X > s + t \mid X > s) = P(X > t).$$

The proof is the same cancellation as before, using the tail $P(X > x) = e^{-\lambda x}$. The consequence is that the expected remaining wait is $\frac{1}{\lambda}$ regardless of how long the wait has already lasted, so elapsed time carries no information about the time still to come.

Counts against waiting times

Poisson and Exponential describe one process from two directions. If events arrive at rate λ , the *count* in a unit window is $\text{Poisson}(\lambda)$ and the *gap* between consecutive events is $\text{Exponential}(\lambda)$. The identity linking them is that zero arrivals in $[0, t]$ is the same event as a wait longer than t :

$$P(N_t = 0) = e^{-\lambda t} = P(X > t).$$

Minimum of exponentials. If $X_i \sim \text{Exponential}(\lambda_i)$ independently, then $\min_i X_i \sim \text{Exponential}(\sum_i \lambda_i)$, and the probability that stream j arrives first is $\lambda_j / \sum_i \lambda_i$. This is the central result for competing arrivals, such as determining which of several order flows reaches the book first.

1.5.7 Uniform Distribution

Definition 1.30 (Continuous Uniform). $X \sim \text{Uniform}(a, b)$ places equal density across an interval:

$$f(x) = \frac{1}{b-a}, \quad F(x) = \frac{x-a}{b-a}, \quad a \leq x \leq b.$$

$$\mu = \frac{a+b}{2} \quad \sigma^2 = \frac{(b-a)^2}{12}$$

Probability for a continuous Uniform is proportional to length, so $P(c < X < d) = \frac{d-c}{b-a}$ for any subinterval. The variance depends only on the width $b-a$ and not on the location, which is the general behavior of variance under a shift.

Do not mix the discrete and continuous forms

The discrete Uniform on $\{1, 2, \dots, n\}$ has mean $\frac{n+1}{2}$ and variance $\frac{n^2-1}{12}$, which is not the continuous formula with $a = 1$ and $b = n$. A fair die has $\mu = 3.5$ and $\sigma^2 = \frac{35}{12} \approx 2.92$. Substituting into $\frac{(b-a)^2}{12}$ would give $\frac{25}{12}$, which is wrong.

Universality of the uniform

For any continuous random variable X with CDF F , the transformed variable $F(X)$ is $\text{Uniform}(0, 1)$. Running this backwards, $F^{-1}(U)$ with $U \sim \text{Uniform}(0, 1)$ has distribution F . This inverse transform is the basis for generating arbitrary distributions from a uniform random number generator, which is the foundation of Monte Carlo simulation. For the Exponential it gives the explicit sampler $X = -\frac{1}{\lambda} \ln(U)$.

1.5.8 Normal (Gaussian) Distribution

Definition 1.31 (Normal). $X \sim \mathcal{N}(\mu, \sigma^2)$ has density

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

The two parameters are exactly its mean and its variance, so the distribution is fully specified by its first two moments.

Two structural properties account for most applications.

Standardization. Any Normal reduces to the standard Normal through $Z = \frac{X-\mu}{\sigma} \sim \mathcal{N}(0, 1)$, which is why a single table of Z values suffices for every Normal question.

Closure under addition. A sum of independent Normals is Normal, with means and variances adding:

$$X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2), X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2) \implies X_1 + X_2 \sim \mathcal{N}(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2).$$

Standard deviations do not add. This is the source of the $\sqrt{T}$ scaling of volatility, since variance grows linearly in time while volatility grows with the square root.

Standard tail masses

Range	Mass inside	Mass outside
$\mu \pm 1\sigma$	68.3%	≈ 1 in 3
$\mu \pm 1.645\sigma$	90%	5% per tail
$\mu \pm 1.96\sigma$	95%	2.5% per tail
$\mu \pm 2\sigma$	95.4%	≈ 1 in 22
$\mu \pm 2.576\sigma$	99%	0.5% per tail
$\mu \pm 3\sigma$	99.7%	≈ 1 in 370

The one-tailed values 1.645 and 2.326 correspond to the 95th and 99th percentiles and are the multipliers underlying standard value-at-risk calculations.

Remark (Why the Normal is the default). The Normal is the maximum entropy distribution among all distributions on the real line with a given mean and variance. Constraining only the first two moments and otherwise assuming as little as possible produces the Normal, which is the formal version of the claim that it encodes minimal structural assumptions. The Central Limit Theorem gives the second, more operational reason for its ubiquity.

Theorem 1.32 (Central Limit Theorem). *Let $X_1, \dots, X_n$ be i.i.d. with finite mean μ and finite variance σ^2 . Then the standardized sample mean converges in distribution to a standard Normal:*

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} \mathcal{N}(0, 1)$$

The standardization is essential to the statement. The sample mean $\bar{X}_n$ on its own converges to the constant μ by the law of large numbers, and its variance σ^2/n shrinks to zero, so it is the rescaled deviation that stabilizes into a Normal shape rather than the mean itself.

Where the CLT fails

The common rule that $n \geq 30$ suffices is a heuristic, not a theorem, and it breaks in three ways that matter for trading:

- **Finite variance is required.** Heavy-tailed distributions with infinite variance, such as the Cauchy, never converge. The sample mean of Cauchy draws has the same Cauchy distribution as a single draw, so averaging buys nothing.
- **Skewness slows convergence.** Strongly asymmetric or rare-event distributions may need n in the thousands before the Normal approximation is usable in the tails, which is exactly where risk questions live.
- **Independence is required.** Financial returns are volatility clustered and serially dependent, so effective sample size is far below nominal sample size.

Empirical asset returns are leptokurtic, meaning excess kurtosis above the Normal value of 3, so extreme moves occur far more often than a Gaussian model predicts.

1.5.9 Lognormal Distribution

Definition 1.33 (Lognormal). X is lognormal when $\ln X \sim \mathcal{N}(\mu, \sigma^2)$, so $X = e^Y$ for a Normal Y . Its support is $x > 0$, and

$$\mathbb{E}[X] = e^{\mu + \sigma^2/2}, \quad \text{median}(X) = e^\mu, \quad \text{Var}(X) = (e^{\sigma^2} - 1) e^{2\mu + \sigma^2}.$$

The lognormal is the standard model for asset prices, for two structural reasons. Prices are bounded below by zero while a Normal would assign positive probability to negative prices, and returns compound multiplicatively, so the logarithm of the price is a sum of log returns and therefore Normal by the same CLT argument.

The mean sits above the median because the distribution is right-skewed. Exponentiating a Normal stretches the upper values far more than the lower ones, producing a long right tail. The median only tracks the middle draw, which is e^μ , but the mean is dragged upward by that tail to $e^{\mu + \sigma^2/2}$. The two differ by the factor $e^{\sigma^2/2}$, which sits at 1 when there is no variability and grows as the volatility σ increases. More spread in the log means a heavier right tail and a wider gap between the average outcome and the typical one.

This gap is a concrete case of a general rule: averaging after applying a curved function is not the same as applying the function to the average. Here the function is e^x , which curves upward, so $\mathbb{E}[e^Y]$ comes out larger than $e^{\mathbb{E}[Y]}$. The excess is exactly the $e^{\sigma^2/2}$ term. The same correction reappears in the drift of geometric Brownian motion, the standard continuous-time price model, for the same reason.

1.5.10 Reference Tables

Discrete distributions

Distribution	Cue in the problem	PMF	Mean	Variance	Support
Bernoulli(p)	one trial, success or failure	$p^x(1-p)^{1-x}$	p	$p(1-p)$	$\{0, 1\}$
Binomial(n, p)	successes in n fixed trials	$\binom{n}{k} p^k (1-p)^{n-k}$	np	$np(1-p)$	$0, \dots, n$
Hypergeom.	successes, no replacement	$\frac{\binom{K}{k} \binom{N-K}{n-k}}{\binom{N}{n}}$	np	$np(1-p) \frac{N-n}{N-1}$	$0, \dots, n$
Geometric(p)	trials until 1st success	$(1-p)^{k-1} p$	$\frac{1}{p}$	$\frac{1-p}{p^2}$	$1, 2, \dots$
NegBin(r, p)	trials until r -th success	$\binom{k-1}{r-1} p^r (1-p)^{k-r}$	$\frac{r}{p}$	$\frac{r(1-p)}{p^2}$	$r, r+1, \dots$
Poisson(λ)	count at a rate, no trial count	$e^{-\lambda} \frac{\lambda^k}{k!}$	λ	λ	$0, 1, 2, \dots$
Uniform $\{1, n\}$	equally likely integers	$\frac{1}{n}$	$\frac{n+1}{2}$	$\frac{n^2-1}{12}$	$1, \dots, n$

Continuous distributions

Distribution	Cue in the problem	PDF	Mean	Variance	Support
Uniform(a, b)	equally likely over a range	$\frac{1}{b-a}$	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$	$[a, b]$
Exponential(λ)	wait until next arrival	$\lambda e^{-\lambda x}$	$\frac{1}{\lambda}$	$\frac{1}{\lambda^2}$	$x \geq 0$
Normal(μ, σ^2)	sum of many small effects	$\frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/2\sigma^2}$	μ	σ^2	$\mathbb{R}$
Lognormal(μ, σ^2)	multiplicative growth	$\frac{1}{x\sqrt{2\pi\sigma^2}} e^{-(\ln x - \mu)^2/2\sigma^2}$	$e^{\mu + \sigma^2/2}$	$(e^{\sigma^2} - 1) e^{2\mu + \sigma^2}$	$x > 0$

Moment generating functions

The MGF is $M_X(t) = \mathbb{E}[e^{tX}]$. It determines the distribution uniquely, generates moments through $\mathbb{E}[X^n] = M_X^{(n)}(0)$, and multiplies across independent sums, which is the fastest proof that Binomials and Poissons and Normals are each closed under addition.

Distribution	$M_X(t)$	Distribution	$M_X(t)$
Bernoulli(p)	$1 - p + pe^t$	Poisson(λ)	$e^{\lambda(e^t - 1)}$
Binomial(n, p)	$(1 - p + pe^t)^n$	Exponential(λ)	$\frac{\lambda}{\lambda - t}, t < \lambda$
Geometric(p)	$\frac{pe^t}{1 - (1 - p)e^t}$	Normal(μ, σ^2)	$e^{\mu t + \sigma^2 t^2 / 2}$
NegBin(r, p)	$\left(\frac{pe^t}{1 - (1 - p)e^t} \right)^r$	Lognormal	undefined for $t > 0$

The lognormal entry is not an oversight. Its right tail is heavy enough that $\mathbb{E}[e^{tX}]$ diverges for every $t > 0$, which is a compact example of a distribution that has all moments yet no MGF.

How the families connect

From	Relationship	To
Bernoulli	sum of n i.i.d. trials	Binomial
Bernoulli	count trials to 1st success	Geometric
Geometric	sum of r i.i.d. waits	Negative Binomial
Binomial	$n \rightarrow \infty, p \rightarrow 0, np = \lambda$ fixed	Poisson
Binomial	n large, $np(1 - p)$ large	Normal
Poisson	gap between arrivals	Exponential
Geometric	continuous time limit	Exponential
Exponential	minimum of independent draws	Exponential (rates add)
Normal	exponentiate	Lognormal
Any distribution	standardized mean, finite variance	Normal (CLT)
Any continuous	apply own CDF	Uniform(0, 1)

Memorylessness is the thread through the waiting-time column. The Geometric and the Exponential are the unique discrete and continuous distributions with that property, which is why both admit one-line recursive expectation arguments.

1.5.11 Select Worked Problems

Problem 1.36

A basketball player makes 70% of free throws. In 10 attempts, what's $P(\textit{exactly 8 makes})$?

Solution. Blah

2 Brainteasers

Problem 2.1 [Easy]

Five pirates need to divide 100 Gold Coins. Pirates have a hierarchy, from Level 1 to level 5. The highest level pirate is the leader. The leader proposes a plan to distribute the gold and all the pirates vote on it (including the leader). If at least 50% of the pirates agree on the plan, the gold is split according to the proposal. If not, level-5 pirate is kicked from the ship, and the level-4 pirate now proposes a new plan. This process continues until a proposal is accepted. All pirates are extremely smart and extremely greedy. How does level-5 Pirate divide the treasure?

Solution. We approach this problem game-theoretically and dynamically:

- 2 pirates → Pirate 2 proposes 100 coins for themselves and zero for Pirate 1
- 3 pirates → Pirate 3 bribes Pirate 1 with one coin and keeps 99 for themselves, which they accept because they get zero coins if Pirate 3 is booted off.
- 4 pirates → Pirate 4 bribes Pirate 2 with one coin and keeps 99 for themselves, in the same manner as the 3-pirate case.
- 5 pirates → Pirate 5 bribes pirates 1 and 3 with one coin each, keeping 98 for themselves.

Thus, the final division is [98, 0, 1, 0, 1].

Problem 2.2 [Easy]

An explorer travels 1 mile South, 1 mile East, then 1 mile North, and finds themselves back at the starting point. The explorer spots a bear, what is the color of the bear?

Solution. The bear must be white. There are a few relevant scenarios that could have resulted in the explorer starting back where they began. In the first case, they started at the North pole, then travelling South equates to travelling directly away from the Pole. They can then travel *any* distance East or West, the following mile back North will return them to their destination. There are no bears in the South Pole so we omit this, though the same logic applies (with the Eastward movement equivalent to spinning in place or not moving at all).

There are other solutions near the South Pole. If the traveler's initial one-mile journey South ends on a latitude circle with a circumference of $\frac{1}{n}$ ($n \in \mathbb{Z}^+$) miles, walking one mile East will complete n full rotations around the pole and leave them exactly where they started walking East, meaning the following mile North would place them in the same starting position.

Problem 2.3 [Easy]

There is a light bulb inside a room and three switches outside. All switches are currently in off state and only one switch controls the light bulb. You may turn any number of switches on or off any number of times you want. How many times do you need to go into the room to figure out which switch controls the light bulb?

Solution. We only need to enter the room once. The first switch is flipped and left on for enough time that the bulb becomes hot. It is then flipped off and the second switch is flipped, we then immediately enter the room. This leads to three cases:

1. Bulb is hot → the light must be off and the first switch controls the light

2. Bulb is on $\rightarrow$ the second switch controls the light
3. Bulb is off and cold $\rightarrow$ the third switch controls the light

If there were four switches, we can still infer the switch in one entry by turning on both switches 1 and 2 for a long time, then switching off switch 2 and turning on switch 3 before entering the room. This results in the following cases:

1. Bulb is on and hot $\rightarrow$ Switch 1 controls the light
2. Bulb is on and cool $\rightarrow$ Switch 3 controls the light
3. Bulb is off and hot $\rightarrow$ Switch 2 controls the light
4. Bulb is off and cool $\rightarrow$ Switch 4 controls the light

The extension to four bulbs worked because we had two binary variables (illumination and bulb temperature) which creates 4 distinct states, of which we were only using 3 in the 3-bulb case.

Problem 2.4 [*Easy*]

How do you place 50 good candies and 50 rotten candies in two boxes such that if you choose a box at random and take out a candy at random, it better be good!

We need to maximize the probability of getting a good candy when selecting a random box and a random candy from it.

Solution. On one extreme, we can place all of the good candies in one box with all of the bad candies in another. This gives us $\mathbb{E}[G] = 0.5$. Thus, we should find solutions that better than 50%. We can beat this rather simply, by placing one good candy in one box and the other 49 in the other box with all of the bad candies. This results in:

$$\mathbb{E}[G] = 0.5 * \frac{1}{1} + 0.5 * \frac{49}{99} \approx 0.747$$

3 Code

3.1 SQL Patterns (PostgreSQL)

Reference for recurring query shapes. Each entry states when to reach for it and the mechanism that makes it work.

Logical processing order

A query is evaluated in this order, regardless of written order:

FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.

Two consequences drive most of the patterns below. **WHERE** runs before aggregation, so it cannot see aggregates or **SELECT** aliases. Window functions are computed at the **SELECT** step, so they also cannot appear in **WHERE** and must be wrapped in a subquery or CTE to filter on.

3.1.1 Filtering and Joins

WHERE vs HAVING. **WHERE** filters individual rows before grouping. **HAVING** filters whole groups after aggregation and is the only place an aggregate predicate is legal. A filter on a plain (non-aggregated) column belongs in **WHERE** so rows are dropped before the grouping work happens. Reserve **HAVING** for conditions like **COUNT(*) > 5**.

```
1 SELECT department, COUNT(*) AS headcount
2 FROM employees
3 WHERE active = true           -- row filter, before grouping
4 GROUP BY department
5 HAVING COUNT(*) > 5;         -- group filter, on an aggregate
```

Listing 1: WHERE prunes rows, HAVING prunes groups

Joins.

- **INNER:** rows with a match on both sides.
- **LEFT / RIGHT:** keep all rows from one side, NULL-fill the other.
- **FULL OUTER:** keep unmatched rows from both sides.
- **CROSS:** Cartesian product, every left row against every right row.
- **SELF:** a table joined to itself under an alias (employee to manager, row to prior row).
- **Semi-join (EXISTS, IN):** keep left rows with at least one match. No row duplication, no right-side columns.
- **Anti-join (NOT EXISTS, or LEFT JOIN ... WHERE right.key IS NULL):** keep left rows with no match.

3.1.2 Window Functions

Every window function shares the same shape: **f() OVER (PARTITION BY ... ORDER BY ...)**. **PARTITION BY** splits the rows into independent groups, **ORDER BY** sets the order within each group, and the function is evaluated per row without collapsing the result the way **GROUP BY** does. All of them are computed at the **SELECT** step, so to filter on their output you wrap them in a subquery or CTE.

Rank-then-filter with ROW_NUMBER(). Use this for top- N per group when you need to keep the full row. GROUP BY would collapse the other columns away. Number rows inside each partition, then filter on that number in an outer query, since the window function cannot live in WHERE.

Choose the ranker by tie policy: ROW_NUMBER assigns distinct numbers and breaks ties arbitrarily, RANK repeats a rank and leaves gaps, DENSE_RANK repeats with no gaps.

```

1 SELECT department, name, salary
2 FROM (
3     SELECT department, name, salary,
4           ROW_NUMBER() OVER (PARTITION BY department
5                               ORDER BY salary DESC) AS rn
6     FROM employees
7 ) ranked
8 WHERE rn = 1;

```

Listing 2: Highest-paid employee per department

Offset access with LAG() and LEAD(). These read a value from another row relative to the current one, without a self-join. LAG(col, k, default) returns the value k rows earlier in the ordered partition, LEAD looks k rows ahead. Both default to an offset of $k = 1$ and return NULL past the partition edge unless a fill default is supplied. The ORDER BY inside OVER defines what “previous” means and is mandatory. PARTITION BY resets the sequence at each group boundary, so a lag never bleeds from one partition into the next.

The dominant use is a row-to-row comparison: a delta, a period-over-period change, or a return. Dividing the current value by its lag gives a growth factor.

```

1 SELECT ticker, trade_date, close,
2        close / LAG(close) OVER (PARTITION BY ticker
3                                ORDER BY trade_date) - 1 AS
4                                daily_return
5 FROM prices;

```

Listing 3: Daily returns per ticker with LAG

The first row of each partition has no predecessor, so its LAG is NULL and the derived return is NULL. That is correct here, since there is no prior price to compare against. LEAD is the same tool aimed forward, used below for gap detection, where a jump between an id and the next present id marks a break in a run.

Running and rolling aggregates. An ordinary aggregate gains a window when you attach OVER (... ORDER BY ...). With an ORDER BY and no explicit frame, the frame defaults to everything from the partition start up to the current row, giving a running (cumulative) total. Add an explicit ROWS BETWEEN frame to get a fixed moving window instead.

```

1 SELECT ticker, trade_date, close,
2        AVG(close) OVER (PARTITION BY ticker ORDER BY trade_date
3                          ROWS BETWEEN 19 PRECEDING AND CURRENT ROW) AS
4                          ma_20,
5        SUM(volume) OVER (PARTITION BY ticker ORDER BY trade_date) AS
6                          cum_volume
7 FROM prices;

```

Listing 4: 20-day moving average and running volume

`ma_20` is the current row plus the 19 before it, a 20-row moving average, and `cum_volume` uses the default frame for a running total. These are the SQL forms of a pandas `.rolling(20).mean()` (fixed ROWS frame) and `.expanding().sum()` (running total from the start).

One subtlety: the default frame is `RANGE UNBOUNDED PRECEDING`, which includes every row tied on the `ORDER BY` key, not just rows physically up to the current one. When ties on the order key exist and you want a row-precise window, state `ROWS BETWEEN` explicitly rather than relying on the default.

Gaps and islands. An *island* is a maximal run of consecutive values. The trick: subtract a dense row number from the value. Across a consecutive run both sides increase by one per step, so the difference stays constant, and it jumps exactly at a gap. Group by the difference to collapse each run.

```
1 WITH numbered AS (  
2     -- DISTINCT first so duplicate ids do not desync the row number  
3     SELECT DISTINCT log_id,  
4         ROW_NUMBER() OVER (ORDER BY log_id) AS rn  
5     FROM logs  
6 )  
7 SELECT MIN(log_id) AS island_start,  
8        MAX(log_id) AS island_end,  
9        COUNT(*)     AS length  
10 FROM numbered  
11 GROUP BY log_id - rn  
12 ORDER BY island_start;
```

Listing 5: Islands of consecutive ids

The `DISTINCT` is load-bearing. If `log_id` repeats, `rn` still advances on the duplicate while the value does not, so `log_id - rn` shifts and splits an island that should stay whole. For consecutive *dates* rather than integers, offset by an interval: `log_date - rn * INTERVAL '1 day'` is constant within a run.

```
1 SELECT log_id + 1 AS missing_from,  
2        next_id - 1 AS missing_to  
3 FROM (  
4     SELECT log_id,  
5         LEAD(log_id) OVER (ORDER BY log_id) AS next_id  
6     FROM logs  
7 ) t  
8 WHERE next_id - log_id > 1;    -- values missing between the two present  
   ids
```

Listing 6: Gaps via LEAD

3.1.3 Aggregation

Conditional aggregation (pivot). Fold a categorical column into one row of counts by wrapping a CASE in an aggregate. Rows the CASE does not match contribute 0 to SUM. In Postgres the FILTER clause does the same thing and reads more cleanly.

```
1 SELECT
2     inspection_type,
3     COUNT(*) FILTER (WHERE risk_category IS NULL)           AS no_risk,
4     COUNT(*) FILTER (WHERE risk_category = 'Low Risk')      AS low_risk
5     ,
6     COUNT(*) FILTER (WHERE risk_category = 'Moderate Risk') AS moderate_risk,
7     COUNT(*) FILTER (WHERE risk_category = 'High Risk')     AS high_risk,
8     COUNT(*)                                                AS total
9 FROM sf_restaurant_health_violations
10 GROUP BY inspection_type
11 ORDER BY total DESC;
```

Listing 7: Pivot risk categories to columns

The SUM(CASE WHEN ... THEN 1 ELSE 0 END) form is equivalent and portable across engines. COUNT ignores NULL, so COUNT(CASE WHEN cond THEN 1 END) (no ELSE) also counts only the matches.

3.1.4 NULL Handling

COALESCE and NULLIF. COALESCE(a, b, c, ...) returns the first non-NULL argument, evaluated left to right, or NULL if all are NULL. The common use is a fallback that keeps a NULL from poisoning arithmetic: salary + commission is NULL whenever commission is, so write salary + COALESCE(commission, 0). It also supplies a default down a priority list, as in COALESCE(nickname, first_name, 'Unknown'). It is ANSI standard, unlike the vendor-specific IFNULL (MySQL) and ISNULL (SQL Server), and it accepts any number of arguments.

NULLIF(a, b) is the mirror image: it returns NULL when a = b, otherwise a. Its standard use guards division, x / NULLIF(y, 0) yields NULL instead of raising a divide-by-zero when y is 0.

```
1 SELECT
2     order_id,
3     revenue / NULLIF(quantity, 0) AS unit_price,      -- NULL, not an
4     COALESCE(discount, 0)         AS discount,        -- NULL discount
5     COALESCE(nickname, first_name) AS display_name
6 FROM orders;
```

Listing 8: NULLIF guards division, COALESCE supplies defaults

A subtlety with aggregates: SUM, AVG, and COUNT(col) skip NULLs. So AVG(commission) averages only the non-NULL rows, while AVG(COALESCE(commission, 0)) counts the NULLs as zeros and enlarges the denominator. Decide which you mean before wrapping the column.

Gotchas.

- **NOT IN with NULLs:** if the subquery returns any NULL, `x NOT IN (...)` is never true and the query returns zero rows. `x NOT IN (1, NULL)` expands to `NOT (x=1 OR x=NULL)`, and the NULL comparison poisons the result to unknown. Use `NOT EXISTS`, which is null-safe.
- **Integer division truncates:** `5 / 2` yields 2 because the fractional part is discarded. Cast one operand: `5::numeric / 2` or `5.0 / 2`. Bites hardest in ratio and percentage columns.
- **COUNT(col) vs COUNT(*):** the former skips NULLs in `col`, the latter counts every row. A common source of off-by-*N* errors.

3.2 Python Patterns

Good solutions to interview problems are often determined by the choice of container and knowledge of which built-in primitives express the required operation.

Containers and cost profiles

Selecting a container is selecting a cost profile. That decision precedes the algorithm and frequently dominates the runtime.

	Ordered	Mutable	Lookup	Use
<code>list</code>	yes	yes	$O(n)$	ordered sequence, duplicates permitted
<code>tuple</code>	yes	no	$O(n)$	fixed record, or a dictionary key
<code>set</code>	no	yes	$O(1)$	membership, deduplication
<code>dict</code>	insertion	yes	$O(1)$	mapping from key to value

Membership cost

`x in some_list` is $O(n)$. `x in some_set` is $O(1)$ on average. A membership test inside a loop over n elements is therefore $O(n^2)$ against a list and $O(n)$ against a set. Converting the list to a set once, before the loop, removes a factor of n . This substitution accounts for a large share of naive-to-optimal improvements.

Hashability. Set elements and dictionary keys must be hashable, which in practice requires immutability. A `tuple` is a valid key. A `list` is not, since mutation would invalidate its position in the hash table. Converting a list to a tuple to form a composite key is therefore a standard construction, as in grouping anagrams by their sorted letters.

```
1 seen = set()
2 seen.add((row, col))           # legal, tuple is hashable
3 # seen.add([row, col])         # TypeError: unhashable type
```

Listing 9: Tuple as a composite key

Dictionary access. Counting, grouping, and seen-before problems reduce to a dictionary. Three access forms differ in their treatment of a missing key.

```
1 d[k]                           # raises KeyError if k is absent
2 d.get(k, default)              # returns default instead of raising
3 d.setdefault(k, [])           # inserts default if absent, then returns it
4
5 for k in d:                    # iterates over keys
6 for k, v in d.items():         # iterates over pairs
```

Listing 10: Dictionary access

Built-in functions

An accumulator loop is often a built-in written out longhand. The following cover most cases.

Built-in	Expresses
<code>len, sum, min, max</code>	size and aggregation
<code>sorted</code>	ordering, parameterized by key and reverse
<code>any(...)</code>	existential quantifier: at least one element satisfies p
<code>all(...)</code>	universal quantifier: every element satisfies p
<code>enumerate</code>	index paired with value
<code>zip</code>	parallel iteration over several sequences
<code>reversed, range</code>	traversal order
<code>abs, round, divmod</code>	numeric utilities

`any` and `all` short-circuit. The expression `any(x < 0 for x in xs)` terminates at the first negative value.

The key argument. `sorted`, `min`, and `max` accept a key function applied to each element. Ranking is performed on the image of that function while the original element is returned. The consequence is that a record can be ranked on one of its fields and recovered whole.

```
1 min(scores)                                # returns a number
2 min(records, key=lambda r: r[1])           # returns the full record
3
4 sorted(words, key=len)                     # rank by length
5 sorted(records, key=lambda r: (-r[1], r[0])) # score desc, name asc
```

Listing 11: Ranking on a field, returning the record

A tuple key is the analogue of a multi-column `ORDER BY`. Negating a single numeric field reverses that field alone. `reverse=True` cannot express this, since it inverts every field simultaneously.

Comprehensions

A comprehension expresses selection and projection in one expression. The form `[f(x) for x in xs if p(x)]` corresponds to selecting $f(x)$ over the elements for which $p(x)$ holds. Four bracket types produce four containers.

```
1 [x*x for x in xs if x > 0]                # list
2 {x*x for x in xs}                         # set, deduplicates
3 {k: v for k, v in pairs}                   # dict
4 (x*x for x in xs)                         # generator, lazy, builds no intermediate list
```

Listing 12: The four forms

The generator form is appropriate when the result is consumed once, for example inside `sum(...)` or `max(...)`, since it avoids materializing a list. Nesting beyond two levels is generally less readable than an explicit loop.

Sorting

- `sorted(xs)` returns a new list. `xs.sort()` sorts in place and returns `None`. The assignment `xs = xs.sort()` binds `None` and fails silently.
- Python's sort is **stable**. Elements comparing equal retain their original relative order. Stability licenses sorting in multiple passes: sorting by the secondary field first and by the primary field second preserves the secondary ordering within each group.
- Sorting a sequence of lists or tuples compares elementwise from the left. The result depends on field order, so an explicit **key** should be supplied whenever the intent is nonobvious.
- Sorting costs $O(n \log n)$, which is negligible at the input sizes these problems use.

The collections module

Four types cover nearly every use. Three are specializations of the dictionary that eliminate the same recurring boilerplate: the check for whether a key already exists before writing to it. The fourth replaces the list when insertion or removal happens at the front.

Counter: frequency. A `Counter` is a dictionary from element to count. It is constructed in one pass over any iterable and answers the question “how many times does each value occur.” Frequency is the most common intermediate object in string and array problems, and constructing one by hand requires an existence check on every element.

```
1 from collections import Counter
2
3 c = Counter("mississippi") # Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})
4 c['s']                    # 4
5 c['z']                    # 0, a missing key reads as zero and does NOT raise
6
7 c.most_common(2)          # [('i', 4), ('s', 4)], top 2 by count
8 c.most_common()          # every pair, ordered by count descending
9 c.total()                # sum of all counts
```

Listing 13: Counter

A missing key evaluates to 0 rather than raising `KeyError`, which is what permits `c[x] += 1` to be written unconditionally. The read does not insert the key, so the counter does not grow through inspection.

`most_common(k)` selects the top k with a heap in $O(n \log k)$. Calling `most_common()` with no argument sorts the whole counter at $O(n \log n)$, so the argument should be supplied whenever k is known.

Counters support multiset arithmetic, which expresses several problems directly.

```
1 a, b = Counter("aabbcc"), Counter("abbbd")
2
3 a + b                    # counts added
4 a - b                    # counts subtracted, results <= 0 are dropped
5 a & b                    # intersection: elementwise minimum
6 a | b                    # union: elementwise maximum
```

Listing 14: Counter arithmetic

Two canonical Counter identities

Anagram test. Two strings are anagrams if and only if their counters are equal, so `Counter(s) == Counter(t)` settles the question in $O(n)$. Sorting both strings answers the same question in $O(n \log n)$.

Sliding-window matching. To locate every permutation of a pattern p inside a text t , maintain a counter over a window of length $|p|$ and compare it against `Counter(p)` after each shift. Incrementing the entering character and decrementing the leaving one updates the window in $O(1)$.

defaultdict: grouping. A `defaultdict` is constructed with a zero-argument factory. Accessing a missing key calls that factory, inserts the result, and returns it. The effect is to remove the initialization branch from every accumulation loop.

```
1 from collections import defaultdict
2
3 groups = defaultdict(list)           # missing key -> []
4 for name, dept in employees:
5     groups[dept].append(name)        # no existence check required
6
7 counts = defaultdict(int)           # missing key -> 0
8 adj     = defaultdict(set)          # missing key -> set(), an
    adjacency list
9 grid    = defaultdict(lambda: defaultdict(int)) # a nested, sparse 2D
    table
```

Listing 15: defaultdict

The factory is any callable taking no arguments. `int()` returns 0, `list()` returns `[]`, and `set()` returns an empty set, which is why the bare type name is passed rather than a call to it. A nested default requires a `lambda`, since `defaultdict(defaultdict(int))` would pass an instance where a factory is expected.

`defaultdict(list)` performs a GROUP BY that omits the aggregation step. Every member of each group survives, which is the property that `ROW_NUMBER() OVER (PARTITION BY ...)` supplies in SQL. Building a graph's adjacency list is the same operation applied to edges.

Reading a defaultdict mutates it

Any access to a missing key inserts it. The expression `if d[k]:` therefore creates `k` with an empty default and the dictionary grows through inspection alone. This corrupts a subsequent iteration over `d.items()` and inflates `len(d)`.

For a read that must not write, use `k in d` or `d.get(k)`. Neither triggers the factory.

The alternative to a `defaultdict` is `d.setdefault(k, []).append(x)` on an ordinary dictionary. It produces the same result and constructs a fresh empty list on every call, including the calls where the key is already present. The `defaultdict` factory runs only on a miss.

deque: a queue. A `deque` is a double-ended queue backed by a doubly linked list of fixed-size blocks. Appending and popping at either end is $O(1)$. The relevant comparison is against a `list`, which stores its elements contiguously, so removing the front element requires shifting every remaining element left by one position.

Operation	list	deque
Append or pop at the right	$O(1)$	$O(1)$
Append or pop at the left	$O(n)$	$O(1)$
Index at position i	$O(1)$	$O(n)$

The costs are complementary. A **list** is the correct stack, since a stack only touches the right end. A **deque** is the correct queue. Random access into a **deque** is a sign that the wrong container was chosen.

```

1 from collections import deque
2
3 q = deque([1, 2, 3])
4 q.append(4)           # right
5 q.appendleft(0)       # left
6 q.pop()              # from the right
7 q.popleft()          # from the left,  $O(1)$  where  $list.pop(0)$  is  $O(n)$ 
8 q.rotate(1)          # shift every element right by one, cyclically

```

Listing 16: deque

Breadth-first search is the standard application. The frontier is processed in insertion order, which requires removal from the front on every iteration of the main loop.

```

1 q = deque([start])
2 seen = {start}
3 while q:
4     node = q.popleft()      # FIFO, so nodes are visited by distance
5     for nxt in adj[node]:
6         if nxt not in seen:
7             seen.add(nxt)
8             q.append(nxt)

```

Listing 17: BFS frontier

Bounded length. A deque constructed with `maxlen=k` discards an element from the opposite end whenever a full deque is appended to. This maintains a rolling window over the last k items with no explicit eviction logic, which is useful for moving averages and for any statistic over a fixed lookback.

```

1 w = deque(maxlen=3)
2 for x in stream:
3     w.append(x)           # the oldest element is evicted automatically
4     if len(w) == 3:
5         print(sum(w) / 3)

```

Listing 18: A rolling window of the last 3 values

namedtuple: a labeled record. A tuple carries no information about what its positions mean, so `r[1]` requires the reader to remember the field order. A **namedtuple** attaches names while remaining a tuple, so it stays immutable, hashable, unpackable, and usable as a dictionary key or a set element.


```

1 from collections import namedtuple
2
3 Student = namedtuple("Student", ["name", "score"])
4 s = Student("Ada", 92.0)
5
6 s.score          # 92.0, and s[1] still works
7 name, score = s  # unpacking is unchanged
8 min(students, key=lambda s: s.score) # the key function reads clearly

```

Listing 19: namedtuple

The cost over a plain tuple is negligible and the gain is that a key function or a sort becomes self-documenting. When mutability is required, `dataclasses.dataclass` provides the equivalent with assignable fields.

Iteration

The following forms eliminate manual index arithmetic, which is the usual source of off-by-one errors.

```

1 for i, x in enumerate(xs):          # index and value together
2 for a, b in zip(xs, ys):            # parallel, halts at the shorter
    sequence
3 for a, b in zip(xs, xs[1:]):        # consecutive pairs
4 for x in reversed(xs):              # reverse traversal without index
    arithmetic
5
6 name, score = record                # tuple unpacking
7 first, *rest = xs                   # starred unpacking

```

Listing 20: Iteration forms

The construction `zip(xs, xs[1:])` yields every adjacent pair and underlies run detection, difference sequences, and monotonicity checks. It is the analogue of `LAG` in SQL.

Problem structure

The four-move skeleton

A typical warm-up problem decomposes into four steps.

1. **Parse** the input into a list of records.
2. **Reduce** the records to a target key: a minimum, a maximum, a count, or the k -th distinct value.
3. **Filter** the records against that key.
4. **Order and format** the output.

Step 2 accounts for most errors. The characteristic failure is fusing steps 2 and 3 into a single pass that mishandles an edge case.

Ranking and ties. The most frequent error in step 2 concerns ties. Whether tied elements share a rank or consume one is the same decision in Python as in SQL.

Tie policy	Python	SQL
Ties consume a rank	<code>sorted(xs)[k-1]</code>	<code>ROW_NUMBER() = k</code>
Ties share a rank	<code>sorted(set(xs))[k-1]</code>	<code>DENSE_RANK() = k</code>

For scores [20, 20, 30], the second lowest is 30. Both elements equal to 20 hold rank 1 and neither occupies rank 2, so `sorted(xs)[1]` evaluates to 20 and is incorrect. The `set` converts `ROW_NUMBER` semantics into `DENSE_RANK` semantics.

Rank then filter. Once the key is identified, it is computed in one pass and the records are filtered against it in a second. The structure mirrors the necessity of wrapping a window function in a CTE, since a window function cannot appear in `WHERE`.

```
1 target = sorted({score for _, score in records})[1]    # rank
2 winners = sorted(name for name, score in records if score == target)
3                                     # filter, then order
```

Listing 21: Rank then filter

Common errors

- `sorted(set(xs))[1]` raises `IndexError` when all values are identical.
- Slicing copies. `xs[a:b]` costs $O(b - a)$, so a slice inside a loop makes the loop quadratic.
- `list.insert(0, x)` and `list.pop(0)` are $O(n)$, since every subsequent element shifts. A `deque` provides $O(1)$ operations at both ends.
- Integer division `//` floors toward negative infinity, so `-7 // 2` evaluates to `-4`. `int(-7 / 2)` truncates toward zero.
- A mutable default argument (`def f(xs=[])`) is created once at definition time and persists across calls. The default should be `None`, with the container constructed inside the func.
- Strings are immutable. Accumulating with `s += c` in a loop is $O(n^2)$. Collect the pieces in a list and apply `".join(parts)`.

3.3 Algorithmic Paradigms

Each paradigm is identified by a recognition cue in the problem statement.

- **Hashing.** Cue: find a pair, count occurrences, seen before, contains a duplicate. Space is traded for $O(1)$ lookup. The canonical construction tests whether the complement of the current element already appears in a map of previously visited elements.
- **Two pointers.** Cue: a sorted sequence, or a pairing problem. Pointers converging from both ends solve “two elements summing to t ” in $O(n)$ with constant extra space. Fast and slow pointers detect a cycle in a linked list.
- **Sliding window.** Cue: a contiguous subarray or substring subject to a constraint. The right edge expands to admit elements and the left edge contracts while the constraint is violated. Each element enters and leaves once, giving $O(n)$.
- **Prefix sums.** Cue: repeated range-sum queries over a static array. Cumulative sums are precomputed once, after which any range query is a difference of two entries. This is the array analogue of `SUM() OVER`.
- **Binary search on the answer.** Cue: the smallest x such that a condition is feasible. Requires a monotone predicate, meaning that feasibility at x implies feasibility at every larger x . The search ranges over the answer space rather than the input.
- **Sorting as preprocessing.** Cue: intervals, scheduling, or grouping equal elements. Sorting converts a global problem into a local one in which each element need only be compared against its neighbor.

Complexity anchors

Operation	Cost
<code>x in set, x in dict</code>	$O(1)$ average
<code>x in list</code>	$O(n)$
<code>list.append</code>	$O(1)$ amortized
<code>list.insert(0,x), list.pop(0)</code>	$O(n)$
<code>deque.appendleft, deque.popleft</code>	$O(1)$
<code>sorted(xs)</code>	$O(n \log n)$
<code>xs[a:b]</code>	$O(b - a)$, copies

3.4 Recursion and Memoization

A recursive function is defined by three obligations.

The recursion contract

1. **Base case.** At least one input for which the answer is returned without a recursive call.
2. **Recursive case.** The answer for input n expressed in terms of the answers for strictly smaller inputs.
3. **Progress.** Every recursive call moves the argument closer to a base case, so the recursion terminates.

Correctness follows by induction. Assuming the recursive call returns the correct answer on the smaller input, verify only that the combination step is correct. Tracing the full call tree by hand is unnecessary.

The call stack. Each pending call occupies a stack frame, so the space cost of a recursion is $O(d)$ where d is the maximum depth. Python imposes a default recursion limit of 1000 frames and raises `RecursionError` beyond it. A recursion whose depth scales with n is therefore unsafe for large n unless it is rewritten as a loop with an explicit stack.

Overlapping subproblems. Naive recursion recomputes identical subproblems. The Fibonacci recurrence $F(n) = F(n-1) + F(n-2)$ generates a call tree with $O(\varphi^n)$ nodes, since $F(n-2)$ is evaluated once by the $F(n-1)$ branch and again directly. Caching each result by its argument collapses the tree to $O(n)$ distinct evaluations.

```
1 from functools import lru_cache
2
3 @cache(maxsize=None)
4 def fib(n):
5     if n < 2:                                # base case
6         return n
7     return fib(n-1) + fib(n-2)               # recursive case
```

Listing 22: Memoization by decorator

Arguments to a memoized function must be hashable, which is the reason memoized state is passed as an integer, a string, or a tuple rather than as a list or a set.

Top-down and bottom-up. Memoized recursion is the top-down form. It computes only the states that are reachable, at the cost of a call stack. Tabulation is the bottom-up form. It fills an array in an order that guarantees every dependency is already resolved, using no stack and no recursion.

```
1 def fib(n):
2     if n < 2:
3         return n
4     prev, curr = 0, 1
5     for _ in range(n - 1):
6         prev, curr = curr, prev + curr    # only two states are ever live
7     return curr
```

Listing 23: The same recurrence, tabulated

The final line illustrates a general reduction. When a recurrence depends only on the previous k states, the table collapses from $O(n)$ space to $O(k)$.

Dynamic programming

Dynamic programming applies when a problem has **optimal substructure**, meaning the optimal solution is composed of optimal solutions to subproblems, together with **overlapping subproblems**, meaning those subproblems recur. Absent overlap, plain divide and conquer suffices. Absent optimal substructure, a greedy or exhaustive method is required.

Formulating a DP

Five questions, answered in order:

1. **State.** What arguments fully determine a subproblem? This is the signature of the function or the index set of the table.
2. **Recurrence.** How is a state's value expressed in terms of smaller states?
3. **Base cases.** Which states are known outright?
4. **Order.** In what order can states be evaluated so that dependencies are resolved first? Top-down recursion answers this automatically.
5. **Answer.** Which state holds the final result?

The complexity is the number of states multiplied by the cost of evaluating one state.

Canonical state definitions. Most problems reuse a small number of state shapes.

State	Meaning	Typical problem
$\text{dp}[i]$	best over the prefix ending at i	maximum subarray, house robber
$\text{dp}[i][j]$	best over two prefixes	edit distance, longest common subsequence
$\text{dp}[i][w]$	first i items, capacity w	0/1 knapsack
$\text{dp}[i][j]$	best over the interval $[i, j]$	matrix chain, palindrome partition

Backtracking

Backtracking is recursion over a search tree with an explicit undo step. It enumerates permutations, subsets, and combinations, and prunes branches that cannot yield a valid solution.

```

1 def solve(path, choices, results):
2     if is_complete(path):
3         results.append(path[:]) # copy, since path is mutated in place
4         return
5     for c in choices:
6         if not is_valid(path, c):
7             continue           # prune
8         path.append(c)         # choose
9         solve(path, remaining(choices, c), results)
10        path.pop()             # undo

```

Listing 24: Backtracking skeleton

The `path[:]` copy is mandatory. Appending `path` itself stores a reference to a list that subsequent mutation will alter, so every recorded result ends up identical.

For unconstrained enumeration, the standard library is preferable to a hand-written recursion.

```

1 from itertools import permutations, combinations, product
2
3 permutations(xs, r)      # ordered arrangements of length r
4 combinations(xs, r)     # unordered selections of length r
5 product(xs, repeat=r)   # r-tuples with repetition, the r-fold
                          # Cartesian power

```

Listing 25: Enumeration without recursion

These correspond exactly to the counting formulas $P(n, r) = n!/(n - r)!$, $\binom{n}{r}$, and n^r .

3.5 Heaps and Priority Queues

A binary heap maintains the minimum of a collection under insertion and deletion at logarithmic cost. Python's `heapq` operates on an ordinary list interpreted as a heap, so no dedicated type is involved.

Operation	Call	Cost
Convert a list in place	<code>heapify(xs)</code>	$O(n)$
Insert	<code>heappush(h, x)</code>	$O(\log n)$
Remove the minimum	<code>heappop(h)</code>	$O(\log n)$
Inspect the minimum	<code>h[0]</code>	$O(1)$
Push then pop	<code>heappushpop(h, x)</code>	$O(\log n)$, one pass

A heap provides no ordering beyond its root. Iterating over the underlying list does not produce sorted output.

Minimum only. `heapq` implements a min-heap exclusively. A max-heap is obtained by negating the keys on insertion and negating again on extraction. For records, push the tuple `(-priority, item)`, since tuples compare lexicographically and the first field dominates.

Top- k under a stream

To retain the k largest of n elements, maintain a min-heap of size k . Each new element is compared against the root, which is the smallest of the current k . If the element is larger, the root is evicted and the element inserted.

The cost is $O(n \log k)$ and the memory is $O(k)$. Sorting the full input costs $O(n \log n)$ and requires all n elements in memory at once, so the heap is superior when $k \ll n$ and is the only option when the input arrives as a stream of unknown length.

```

1 import heapq
2
3 def top_k(stream, k):
4     h = []                                # min-heap holding the k best
5     so far
6     for x in stream:
7         if len(h) < k:
8             heapq.heappush(h, x)
9         elif x > h[0]:                     # h[0] is the weakest survivor
10            heapq.heapreplace(h, x)       # pop the root, then push x
11    return sorted(h, reverse=True)

```

Listing 26: The k largest elements

When the whole input already fits in memory, the library provides this directly as `heapq.nlargest(k, xs)` and `heapq.nsmallest(k, xs)`, both of which accept a `key` argument.

Two heaps for a running median. Maintain a max-heap over the lower half of the values and a min-heap over the upper half, rebalancing after each insertion so their sizes differ by at most one. The median is then read from the root of the larger heap, or is the mean of the two roots when the sizes are equal. Insertion costs $O(\log n)$ and the median query is $O(1)$.

3.6 Sliding Window

A sliding window is a contiguous span of a sequence, delimited by two indices ℓ and r , together with a state summarizing the elements inside it. The window advances by admitting the element at r and evicting the element at ℓ . The technique applies when the state can be updated in $O(1)$ under those two operations, so that recomputing it from scratch is never necessary.

The recognition cue is a request for a *contiguous* subarray or substring subject to a constraint. Contiguity is required. A problem about subsequences or subsets admits reordering and belongs to a different paradigm.

Why the cost is linear

The amortization argument

Both indices are monotone. r advances once per iteration of the outer loop and ℓ never decreases, so each index takes at most n values over the entire run. The inner **while** loop is therefore bounded by n increments *in total* rather than n increments per outer iteration. The nesting is syntactic. The work is $O(n)$: every element is admitted exactly once and evicted at most once. The naive alternative enumerates all $O(n^2)$ spans and pays the cost of the state on each.

The precondition is that ℓ may safely move forward. This holds when the constraint is **monotone in the window**: if a window violates the constraint, every window containing it also violates it. Under that property, once $[\ell, r]$ fails, no window starting before ℓ and ending at r or later can succeed, so the discarded prefixes are provably useless.

Fixed-size windows

When the length k is given, there is no contraction step. The window is initialized once and then translated, one position at a time, by a paired admit and evict.

```
1 def max_window_sum(xs, k):
2     if len(xs) < k:
3         return None
4     total = sum(xs[:k])           # the window is built once, in O(k)
5     best = total
6     for r in range(k, len(xs)):
7         total += xs[r] - xs[r-k] # admit xs[r], evict xs[r-k]
8         best = max(best, total)
9     return best
10
11 max_window_sum([2, 1, 5, 1, 3, 2], 3) # 9, from the span [5, 1, 3]
```

Listing 27: Maximum sum over any window of length k

The single line `total += xs[r] - xs[r-k]` is the whole method. It replaces an inner loop of length k , reducing $O(nk)$ to $O(n)$. The index $r - k$ is the element leaving the window, since the window covering positions $r - k + 1$ through r has just replaced the one covering $r - k$ through $r - 1$.

A counter as the window state. The state need not be a number. Matching a pattern's permutations, described earlier under **Counter**, carries a frequency table as the state and compares it against a fixed target.


```

1 from collections import Counter
2
3 def find_anagrams(t, p):
4     k = len(p)
5     need = Counter(p)
6     have = Counter(t[:k])
7     out = [0] if have == need else []
8     for r in range(k, len(t)):
9         have[t[r]] += 1           # admit
10        have[t[r - k]] -= 1       # evict
11        if have[t[r - k]] == 0:
12            del have[t[r - k]]    # see the note below
13        if have == need:
14            out.append(r - k + 1)  # the window's left index
15    return out
16
17 find_anagrams("cbaebabacd", "abc") # [0, 6]

```

Listing 28: Every starting index at which a permutation of p occurs in t

Each shift performs a constant amount of work and one dictionary comparison over an alphabet of bounded size, giving $O(n)$.

Zero-valued keys in a Counter

Decrementing to zero leaves the key present with value 0. Two consequences follow, and they behave differently.

Equality. On Python 3.10 and later, `Counter.__eq__` compares counts over the union of the keys and treats a missing key as zero, so `Counter({'a':1,'b':0}) == Counter({'a':1})` evaluates to `True`. On Python 3.9 and earlier the class inherits `dict.__eq__`, which compares key sets, and the same expression is `False`.

Size. `len(counter)` counts keys on every version. A key held at zero still contributes to `len`, so any window that tests the number of *distinct* elements must delete the key on reaching zero.

Deleting the exhausted key satisfies both, on every version.

Variable-size windows

When the length is unknown, r expands the window on every iteration and an inner loop moves ℓ forward. The two problem shapes differ in the test governing that inner loop and in the point at which the answer is recorded.

	Longest valid window	Shortest sufficient window
Inner loop	<code>while</code> invariant is broken	<code>while</code> condition holds
Effect	restore validity	test whether ℓ can be given up
Record at	after the loop, on a valid window	inside the loop, before evicting
Answer	<code>max(best, r - 1 + 1)</code>	<code>min(best, r - 1 + 1)</code>

Longest valid window. The window is repaired after each expansion, so it is valid at the bottom of every outer iteration, which is where its length is measured.

```

1 def longest_unique(s):

```

```

2     seen = set()                                # the state: which characters are in
        the window
3     left = best = 0
4     for right, c in enumerate(s):
5         while c in seen:                        # admitting c would break the
            invariant
6             seen.remove(s[left])                # evict from the left until it would
            not
7             left += 1
8         seen.add(c)                             # admit
9         best = max(best, right - left + 1)      # the window is valid here
10    return best
11
12    longest_unique("abcabcbb")                    # 3, from "abc"
13    longest_unique("pwwkew")                     # 3, from "wke"

```

Listing 29: Longest substring with no repeated character

The eviction loop is written before the admission so that the invariant is never violated inside the set. Admitting first and then repairing is equally correct provided the repair condition is written to match, and the ordering must be chosen once and held to.

```

1     from collections import defaultdict
2
3     def longest_k_distinct(s, k):
4         count = defaultdict(int)                # the state: a frequency table over
            the window
5         left = best = 0
6         for right, c in enumerate(s):
7             count[c] += 1                        # admit
8             while len(count) > k:                # invariant: at most k
                keys
9                 count[s[left]] -= 1
10                if count[s[left]] == 0:
11                    del count[s[left]]            # required, or len(count)
                is wrong
12                left += 1
13                best = max(best, right - left + 1)
14        return best
15
16    longest_k_distinct("eceba", 2)                # 3, from "ece"

```

Listing 30: Longest substring with at most k distinct characters

Here the admission precedes the repair, because the invariant is stated over the window's state rather than over the incoming element. The `del` is what makes `len(count)` equal to the number of distinct characters currently present.

Shortest sufficient window. The inner loop contracts while the target condition is still satisfied, which searches for the smallest left index that keeps it satisfied. The length is recorded before each eviction, since the window may cease to qualify immediately afterward.

```

1     def min_length(xs, t):
2         total = 0
3         left = 0
4         best = float('inf')

```

```

5     for right, x in enumerate(xs):
6         total += x                                # admit
7         while total >= t:                          # the window already
            suffices
8             best = min(best, right - left + 1)    # record, then try to
            shrink
9             total -= xs[left]                    # evict
10            left += 1
11    return 0 if best == float('inf') else best
12
13 min_length([2, 3, 1, 2, 4, 3], 7)    # 2, from [4, 3]
14 min_length([1, 1, 1], 7)            # 0, no window qualifies

```

Listing 31: Shortest subarray with sum at least t , over positive values

`float('inf')` as the initial value of a minimum removes the special case for an empty candidate set. The sentinel is then translated to whatever the problem specifies for “no such window,” which is commonly 0.

When the window does not apply

The monotonicity precondition

`min_length` above is correct only because every element is positive. Positivity makes `total` nondecreasing in the window’s length, so `total >= t` is monotone and a failing prefix can be discarded permanently.

Introduce a negative value and that reasoning collapses. Extending a window may now decrease its sum, and a left index abandoned earlier could belong to the optimal answer. On `[3, -2, 5]` with $t = 6$, the window `[3, -2, 5]` qualifies while neither the prefix `[3, -2]` nor the suffix `[5]` taken alone reveals it by contraction.

The replacement is prefix sums with a hash map, which considers every left endpoint rather than only the ones a monotone rule preserves. The check for a negative value in the constraints is therefore the first thing to perform on any subarray-sum problem.

Common errors

- The window length is `right - left + 1`. Omitting the `+1` is the standard off-by-one, since both endpoints are inclusive.
- Recording the answer at the wrong point. A longest-window problem measures after the repair loop. A shortest-window problem measures inside the contraction loop, before the eviction.
- An `if` where a `while` is required. One eviction restores the invariant only when a single admission can break it by exactly one unit. The general repair is a loop.
- Failing to evict from the state. Advancing `left` without removing `xs[left]` from the set, counter, or running sum leaves the state describing a window that no longer exists.
- Retaining zero-valued keys in a `Counter` or `defaultdict` while testing `len` against a distinct-count bound.
- Recomputing the state over the span, as in `sum(xs[left:right+1])` inside the loop. This restores the $O(n^2)$ cost the window exists to remove.